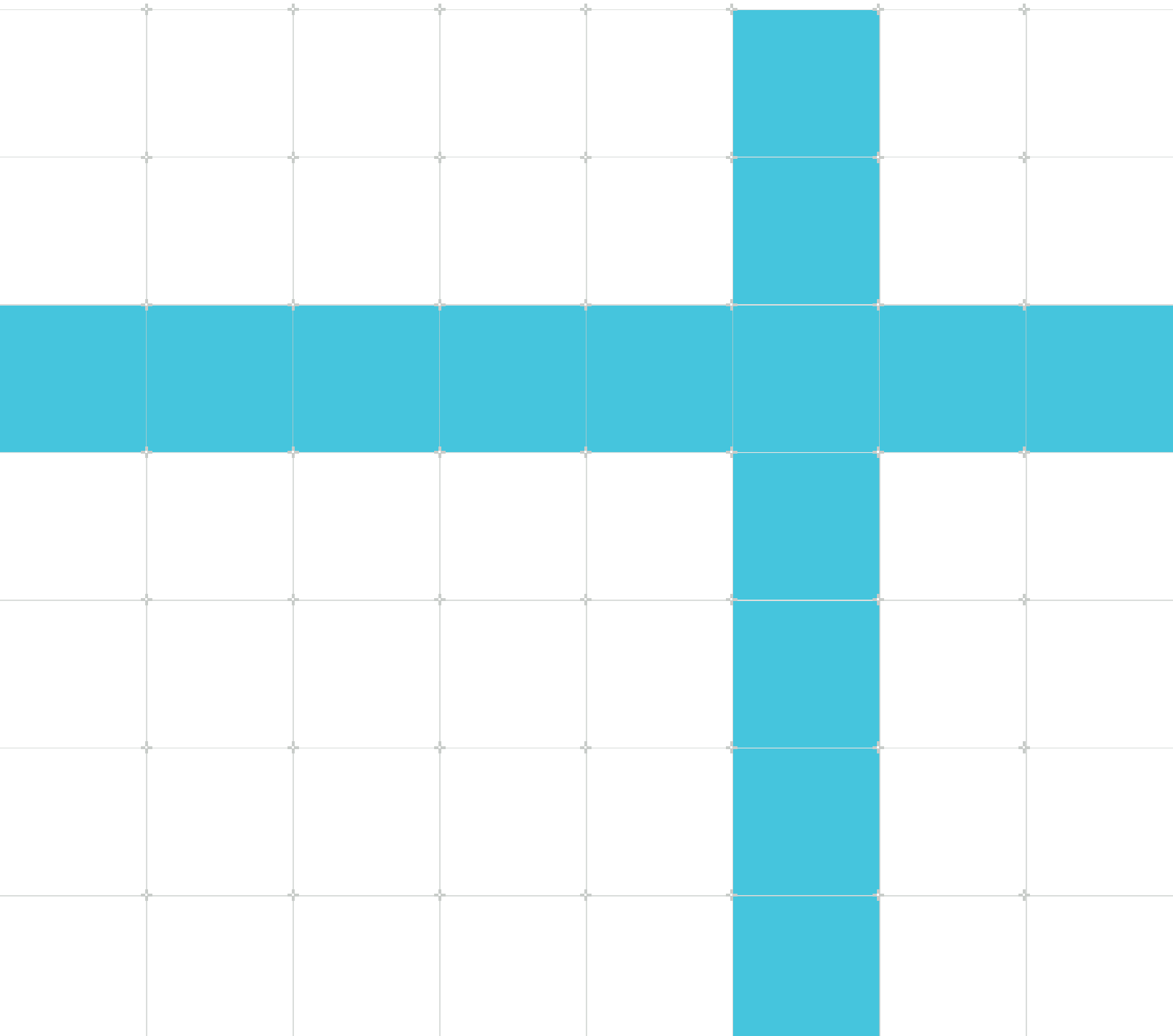
Arm® Mali™ GPU 5th Generation and Later

Revision 1.0

Virtualization Guide

Non-Confidential
Copyright © 2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 01
112001_0100_01_en



Arm® Mali™ GPU 5th Generation and Later Virtualization Guide

This document is Non-Confidential.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.
Arm only permits use of this document if you have reviewed and accepted Arm's Proprietary Notice found at the end of this document.

This document (112001_0100_01_en) was issued on 2026-09-30. There might be a later issue at https://developer.arm.com/documentation/112001

The product revision is 1.0.

See also: Proprietary notice | Product and document information | Useful resources

Start reading

If you prefer, you can skip to the start of the content.

Intended audience

This guide is written for software developers who are developing the software stack for virtualization on top of Graphics Processing Units (GPUs). Typically, the reader is integrating OSes and drivers for a particular platform. This guide provides guidance on how to integrate GPUs into a virtualized platform. It describes the reference code that is used to do this integration.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

This document includes language that can be offensive. We will replace this language in a future issue of this document.

To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on https://support.developer.arm.com.

To provide feedback on the document, fill the following survey: https://developer.arm.com/documentation-feedback-survey.

Contents

1. About this document **9**
1.1 How to read this guide 9

2. Core concepts **10**
2.1 Virtual machine, guest, and driver instance 10
2.2 GPU subinstance 11
2.3 Access windows 11
2.3.1 Open and closed access windows 11
2.4 Time slicing 12
2.5 Yield and handover 13
2.6 Firmware model 13
2.7 Arbiter 13
2.7.1 Arbiter reference implementation and platform policy 14
2.7.2 Fairness versus policy 14
2.8 Hypervisor and external isolation 15
2.9 Partitions and time slicing 15
2.10 How different core virtualization concepts combine 16

3. Virtualization approaches **17**
3.1 Single-partition systems 17
3.1.1 Performance and interference in the shared model 18
3.2 Multi-partition systems 18
3.3 Hybrid systems 19
3.4 System considerations 20
3.4.1 Scheduling policy 20
3.5 What to read next 21

4. System Architecture **22**
4.1 Hardware overview 22
4.2 Software stack 23
4.3 Access Manager 25
4.3.1 Control pages and ownership boundaries 25
4.3.2 Access windows in the Access Manager 26

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.3.3 Arbiter and hypervisor split....26
4.3.4 Interrupt and message paths....27
4.3.5 Cooperative handover....27
4.3.6 Why reset is part of the architecture....28
4.3.7 Time-slicing examples....29
4.3.8 External memory isolation....32
4.3.9 Shared and partition-capable systems....33

5. System Integration....34
5.1 General integration flow....34
5.2 Software component responsibilities....34
5.3 Reference stack and partner adaptation....36
5.4 Bring up the software....37
5.5 Queue and state lifecycle....38
5.6 Non-automotive systems....39
5.7 Automotive systems....39

6. Power management model....40
6.1 Core message....40
6.2 Main software roles....40
6.3 Powered-off behavior....41
6.4 Runtime PM integration....41
6.5 An Arm-style cross-VM approach....42
6.6 Logical and physical power control....42
6.7 Dynamic voltage and frequency scaling and dynamic core scaling....42
6.8 Guest driver assumptions....43
6.9 System-integration implications....43

7. Hypervisor integration....44
7.1 Hypervisor responsibilities....44
7.2 Access-window assignment....44
7.3 Privileged ownership model....45
7.4 Hypervisor and arbiter boundary....45
7.5 Guest-visible device model....45
7.6 Memory translation and isolation....46
7.7 Interrupt routing....46
7.8 Power-management interaction....47

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

7.9 Access-window reassignment....47
7.10 Common integration mistakes....48
7.11 Protected-memory mapping....48

8. Performance and system tuning....50
8.1 Performance tuning considerations....50
8.2 Main sources of overhead....51
8.3 Yield latency and performance....51
8.4 Ownership transfer sequence....52
8.5 Sources of switching overhead....52
8.6 Performance characteristics of shared systems....53
8.7 Platform effects on switching latency....53
8.8 Translation and system overheads....53
8.9 Power management overheads....54
8.10 Performance tuning in partitioned systems....54
8.11 Performance tuning controls....55
8.11.1 Arbitration timeout parameters....55
8.12 Content profiling and workload placement....56
8.13 Example tuning patterns....56
8.13.1 Latency-sensitive guest plus background guest....57
8.13.2 Controlled and open workload environments....57
8.14 Trace-based profiling....58
8.14.1 Trace sources for virtualization profiling....58
8.14.2 Analyzing system behavior with trace data....59
8.14.3 Measuring ownership transfer latency....60
8.14.4 Profile representative content....60
8.15 Architectural performance constraints....61
8.16 Tune virtualization performance....61

9. Reference software stack....62
9.1 Source layout....62
9.2 Main modules....63
9.2.1 panthor.ko....64
9.2.2 panthor_system.ko....65
9.2.3 panthor_arbitration.ko....66
9.2.4 panthor_governor.ko....67
9.3 Integration model....68

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

9.4 Probe and module load order ... 68
9.4.1 Deferred probe behavior ... 69
9.5 Linux reference stack overview ... 69
9.5.1 Initialize the Linux reference stack ... 70
9.5.2 Validate Linux stack initialization ... 70

A. Single OS worked example ... 71
A.1 Scope of the single-OS example ... 71
A.2 Why this environment is useful ... 72
A.3 Single-OS system deployment ... 72
A.4 Initialize a single-OS deployment ... 73
A.5 Validation checklist ... 73
A.6 Relationship to later integration ... 74
A.7 Shared-partition GPU deployment ... 74
A.7.1 Shared-partition GPU device-tree example ... 75
A.7.2 Initialize a shared-partition GPU deployment ... 77
A.7.3 Shared-partition arbitration configuration ... 78
A.8 Multi-partition GPU deployment ... 78
A.8.1 Multi-partition GPU device-tree example ... 79
A.8.2 Initialize a multi-partition GPU deployment ... 81
A.8.3 Example probe messages ... 82
A.8.4 Multi-partition GPU configuration ... 83
A.9 Validate access-window switching ... 85
A.9.1 Access-window switching trace states ... 86

B. pKVM worked example ... 87
B.1 Scope of the pKVM example ... 87
B.2 Why pKVM needs its own worked example ... 87
B.3 pKVM system deployment ... 88
B.4 Bring up and validate a pKVM deployment ... 91
B.5 Access-window assignment and reassignment ... 91
B.6 Guest-visible device model in pKVM ... 91
B.7 Protected-memory handling in pKVM ... 92

C. Porting Guide ... 93
C.1 Scope of porting effort ... 93
C.2 Two porting strategies ... 94

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

C.3 Architectural contracts....94
C.4 Porting interfaces....94
C.4.1 Arbiter-VM protocol....95
C.4.2 Core contracts that must be preserved....95
C.4.3 What the guest side observes....96
C.4.4 What the privileged side observes....97
C.4.5 Porting implication....97
C.4.6 Arbitration-side public interfaces....98
C.5 Porting the guest-side driver model....99
C.5.1 Porting the guest-side driver model....99
C.5.2 Guest-side state handling....100
C.5.3 Guest-side interrupt handling....100
C.5.4 Guest-side protected memory....101
C.6 Porting or replacing the privileged virtualization stack....101
C.6.1 System-control role....102
C.6.2 Arbiter role....102
C.6.3 Governor role....103
C.6.4 VM assignment and repartition roles....103
C.6.5 VM assignment and repartition interfaces....104
C.7 Linux service equivalents for other operating systems....104
C.7.1 Identify operating-system service equivalents....105
C.8 Device tree, platform description, and configuration....105
C.9 RTOS and microkernel considerations....106
C.10 What a partner can change freely....107
C.11 What a partner must not break....107
C.12 Validate a port....108
C.13 Scope of the porting guide....108

D. Arbiter-VM Protocol Appendix....109
D.1 Purpose of the protocol....109
D.2 Versioning....110
D.3 Message directions....110
D.4 Message meanings....110
D.5 Typical protocol flow....112
D.6 Guest-side access-window states....113
D.7 Relationship to access-window interrupts....113

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

D.8 Debugging notes..... 114

Proprietary notice.....115

Product and document information..... 117
Product status.....117
Revision history..... 117
Conventions.....118

Useful resources..... 120

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

1. About this document

This guide explains how to use virtualization in an Arm® Mali™ 5th Generation and Later GPUs.

This guide covers:

- The main virtualization models that can be built around architecture version 9.4, and versions 15 and later GPUs.
- The architectural role of the Access Manager.
- The interaction between the GPU driver, firmware, hypervisor, and arbiter.
- Integration considerations for non-automotive and automotive systems.
- Practical guidance on performance, robustness, and porting.

This guide does not:

- Provide details for every register in the GPU

1.1 How to read this guide

If you are deciding on a system design, start with the core concepts chapter and then the virtualization approaches chapter. If you are already committed to a design and require implementation guidance, read the system architecture and integration chapters.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

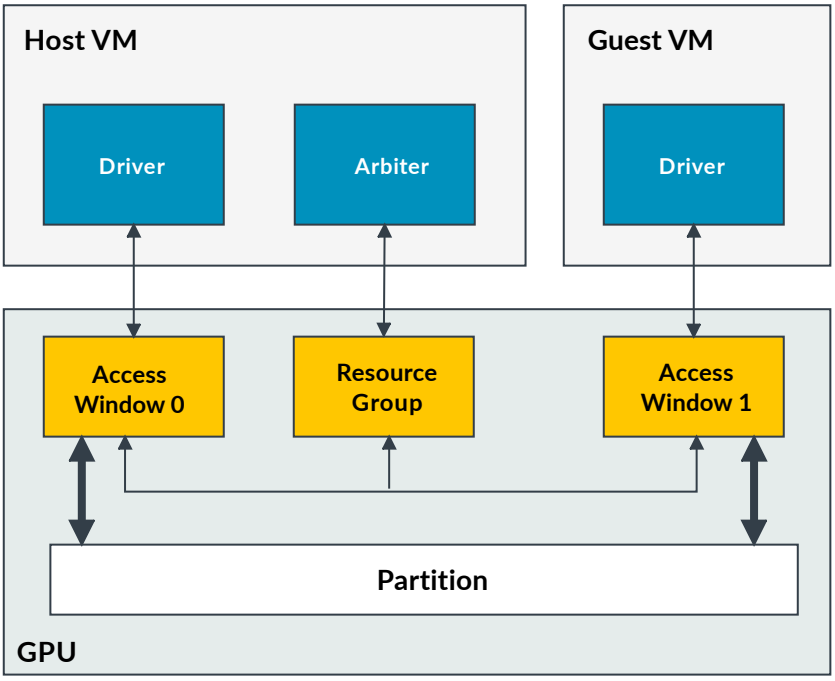
2. Core concepts

The following chapter introduces the terms and mental model that this guide uses.

Many systems use the basic model. The basic model is a shared, single-partition, GPU. Products that support multiple partitions extend this model with additional hardware-isolated execution domains.

The following image shows the different parts of the basic model.

Figure 2-1: VMs and partition



2.1 Virtual machine, guest, and driver instance

The software domain that receives controlled access to the GPU is called the Virtual Machine (VM) or guest.

Within each guest, the most important privileged software component is the GPU kernel driver. This component is the driver instance for that guest. The driver instance is responsible for:

- Programming the GPU control interface that is visible to the guest
- Loading the firmware that runs on the GPU subinstance
- Reacting to interrupts and fault conditions that are visible using its assigned interface
- Participating in yield and restart flows when the arbiter issues an ownership change request
- Interfacing with the user-space to allow applications to submit GPU workloads

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

In most systems, each guest that uses the GPU has its own driver instance. This separation is built into the virtualization design. This separation ensures that the control and data paths for each guest driver are isolated from the control and data paths for another guest driver.

Related information

Arbiter on page 13

2.2 GPU subinstance

A GPU subinstance is the execution GPU hardware of the GPU that a guest can control while its access window is open. From the perspective of the guest, this is the GPU it is driving.

The GPU subinstance concept allows the GPU to be shared between multiple Access Windows in a time-sliced manner. The system moves ownership of the real execution environment between different software domains.

2.3 Access windows

An access window is the control path that is visible to a guest for one GPU driver instance.

The access window:

- Provides the MMIO/control interface that the guest driver uses to access its assigned GPU subinstance
- Provides the guest with a stable interface that can remain mapped into that guest's memory address space

A guest does not require permanent direct ownership of the entire GPU control space. Instead, a guest requires a stable control-plane interface that the arbiter can enable or disable as ownership changes. The access window provides this stable control-plane view.

Using access windows reduces the amount of work required from the hypervisor when GPU subinstance ownership moves between guests. The guest-facing mapping can remain stable, while the Access Manager controls which guest connects to the GPU subinstance at any given time.

This behavior gives the software stack a cleaner ownership model where:

- Guest mappings remain stable across ownership changes
- Interrupt routing can be aligned with access-window ownership
- GPU handover becomes an explicit architectural flow instead of a sequence of ad-hoc remaps and register ownership changes

2.3.1 Open and closed access windows

At the system level, an access window can be open or closed.

These states mean:

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

open

In this state, the guest can interact with the GPU subinstance using the open window.

closed

In this state, the guest retains limited control-path visibility, and cannot continue normal execution using the closed window.

An access window is more than an address range. It is a hardware-controlled ownership boundary.

2.4 Time slicing

Time slicing enables multiple guests to share 1 GPU subinstance. Time slicing changes hardware ownership over time instead of physically partitioning the hardware.

Time slicing is the main virtualization model. The basic behavior is:

- Multiple guests can each have an assigned access window.
- The platform connects only 1 access window to the shared GPU subinstance at a time.
- A software component decides when ownership moves.
- The current owner must stop using the GPU before the next owner takes over.

Each guest can be presented with its own GPU control interface, but the platform serializes ownership of the shared execution environment.

Using time slicing affects the performance of the GPU. To transfer ownership, the arbiter performs these operations:

1. Requests a yield from the current open access window
2. Resets the GPU
3. Open the next access window
4. Resumes or reboots execution for the next owner

Time slicing involves tradeoffs. Some key benefits of time slicing are:

- Better aggregate utilization of one shared GPU
- Less hardware duplication

Key disadvantages of using time slicing are:

- More switching overhead
- More sensitivity to the behavior of guests or a badly tuned arbitration policy

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

2.5 Yield and handover

A yield occurs when the current owner stops controlling the GPU subinstance so that another guest can run.

The handover sequence is:

1. The platform decides the current guest must stop owning the GPU.
2. The platform asks the current owner to yield immediately or when idle.
3. The firmware quiesces execution and preserves the state required to resume later.
4. The current access window is closed and access to the GPU subinstance is revoked.
5. The GPU subinstance resets and a new access window is opened for the next owner.
6. The next guest resumes or starts its execution environment.

Yields are cooperative. The software stack and firmware are expected to participate in an orderly handover. If this cooperation fails, the platform might force recovery. The visible results of forced recoveries can be a loss of work or device-lost style behaviors.

2.6 Firmware model

The firmware is part of the GPU subinstance execution environment. It is not a single high-level shared software service that all guests jointly own.

The firmware:

- Is part of the currently active execution environment.
- Participates directly in scheduling, quiescence, and yield behavior.
- Is tied to the ownership and lifecycle of the GPU subinstance.
- Is booted or warm-resumed every time an access window is opened

Different guests can use different firmware images.

In practice, the firmware is coupled to access-window ownership. When one guest yields and another takes over, the transition changes both the driver-visible registers and the execution state that the firmware manages.

2.7 Arbiter

The arbiter is a software component that decides which access window can access the GPU subinstance. It also decides when to request ownership changes.

The arbiter has the following responsibilities:

- Requesting yield from the current owner
- Selecting the next access window to open

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- Handling timeout and recovery if ownership handover does not complete cleanly

Depending on the platform, the arbiter can be:

- Part of the hypervisor
- A separate privileged software component
- A software service that is associated with a system or safety controller

In the reference software stack, we implement the arbiter as a set of kernel modules that run in the host domain or in a dedicated Virtual Machine (VM). You can move the same functionality into the hypervisor or a safety island, but doing so usually increases certification and integration effort. We recommend that you only make this move if your platform requires it.

2.7.1 Arbiter reference implementation and platform policy

We provide a reference arbiter implementation that uses a simple, fair, round-robin scheduling policy. The arbiter is modularized so that the scheduler is a separate component.

The round-robin behavior means that the scheduler rotates ownership between eligible guests. You can configure this round-robin behavior for:

- When the arbiter begins a switch away from the current owner
- How long a guest is permitted to continue after it has been asked to stop

This reference behavior is intentionally simple. We provide this reference as an example and baseline implementation for the software stack. It does not define the only scheduling policy that we recommend for production systems.

You can add platform-specific behavior to the arbiter. For example:

- VM priority
- Scheduling that is aware of the display or VSync
- Workload asymmetry between guests
- Policies tied to safety, user experience, or product power goals

We do not provide all such platform-specific scheduling implementations as part of the reference stack. Replace or extend the default round-robin scheduler if your system requires different behavior.

2.7.2 Fairness versus policy

The arbiter implements the platform policy for sharing a GPU subinstance between guests.

You can tune the policy for different system goals, for example:

- Reducing latency for interactive workloads
- Improving throughput for shared graphics or compute workloads
- Improving predictability for safety-oriented systems

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The arbiter can decide which guest runs next, how quickly ownership moves, and how much preference one guest receives over another. However, arbitration is still time-sharing of the same physical execution resources. Arbitration can reduce interference, but it cannot provide strong isolation by itself. Strong isolation requires partitioning.

2.8 Hypervisor and external isolation

The hypervisor and the Access Manager have different responsibilities. The Access Manager controls ownership of the GPU control path and the handover between guests. The hypervisor and the surrounding platform are responsible for system-level isolation.

The responsibilities of the hypervisor include:

- Stage-2 memory mapping
- Interrupt routing
- Assignment of access windows to the correct guests
- Ensuring that the security model remains intact

The access window is one part of the larger platform design that includes the hypervisor. Because the platform separates the responsibilities for isolation and ownership of the GPU control path, the access-window control is not the entire virtualization solution.

Placing the arbiter outside the hypervisor does not automatically break guest isolation. The key consideration is the data the software can access. In the intended model, the hardware interfaces that face arbitration are control interfaces for ownership and scheduling. These interfaces do not expose guest GPU workload data.

2.9 Partitions and time slicing

Partitions are hardware-level isolation domains in GPUs with support for physically separated execution resources. Arm AE GPUs support partitions and provide an additional virtualization mechanism that enables multiple GPU subinstances to exist at the same time.

Partitions enable multiple guests to run concurrently on hardware-isolated execution domains.

Partitioning and time slicing are independent mechanisms and can operate at the same time. Partitioning isolates hardware execution resources from each other. Time slicing controls how guests share access over time within one execution domain.

A system can use these mechanisms in the following ways:

Time slicing without partitions
: Multiple guests share one GPU subinstance over time.

Partitions without time slicing
: Each guest has dedicated hardware ownership.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Partitions and time slicing together

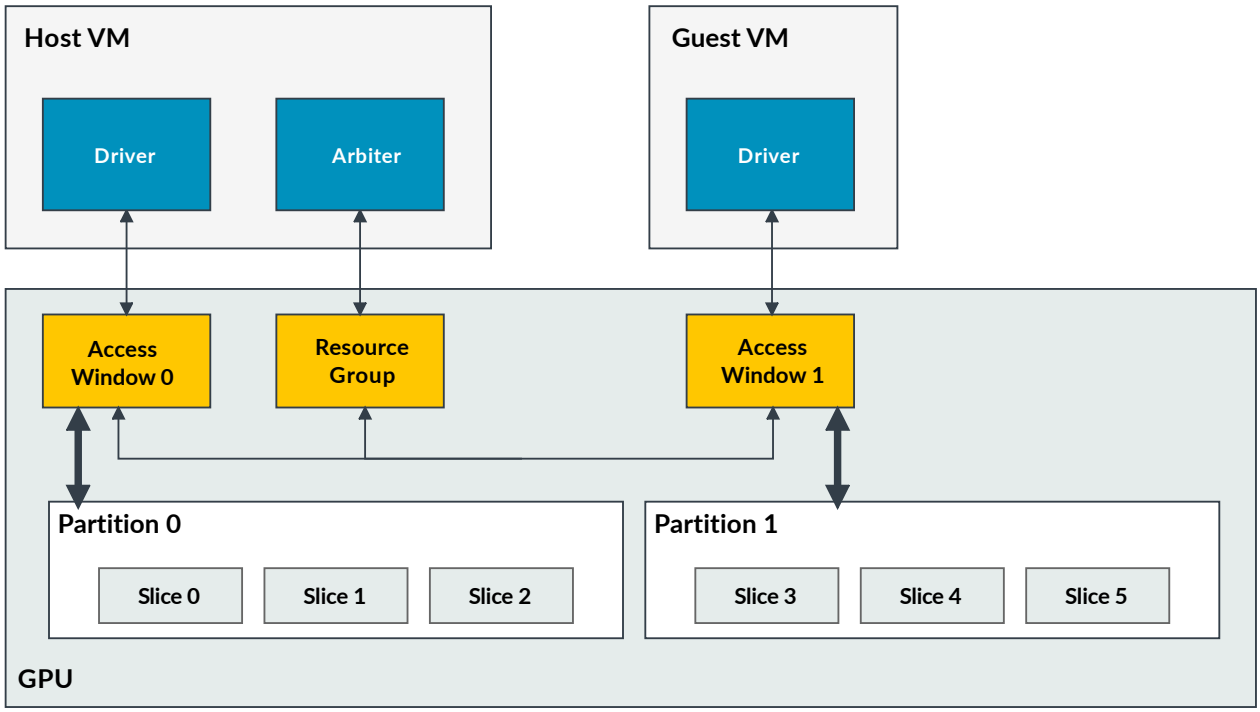
Each partition is hardware-isolated but multiple guests share the partition over time.

For example, a platform with support for both partitions and time slicing can assign different operating systems to different partitions for isolation. At the same time it can use time slicing for multiple guests inside one of those partitions.

Partitioning support affects the design emphasis for a GPU in a system. Time sliced systems emphasize utilization and arbitration. Partition-capable systems emphasize isolation and predictability.

Using partitioning can affect performance at the system level. CPU time, memory bandwidth, and other shared resources can create contention. However, at the GPU execution-domain level, partitions provide a stronger separation boundary than time slicing can.

Figure 2-2: VMs partition for hardware separation



2.10 How different core virtualization concepts combine

The different core concepts combine to affect the driver, access manager, arbiter, firmware, and hypervisor.

Each guest has its own driver instance with an access window visible to the guest. The access manager selects the access window that connects to the GPU subinstance. The arbiter decides when ownership changes. The firmware helps quiesce the active execution environment and hand it over when the arbiter requests it. The hypervisor and platform provide the memory and interrupt isolation that the design requires.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

3. Virtualization approaches

The following chapter describes the main system models that you can build around the virtualization mechanisms that your GPU supports. It also explains how to choose between them for your use cases.

Deciding which model is best for your use case depends on whether guests are sharing the same GPU execution domain or are hardware-isolated. If more than 1 guest must use the same domain, consider how guests share that access over time.

The available model families are:

Single-partition shared systems
: In these systems, guests time slice 1 execution domain.

Multi-partition systems
: In these systems, different guests can run concurrently on separate execution domains.

Hybrid systems
: These systems use hardware separation and time-sliced sharing together.

3.1 Single-partition systems

Single-partition systems are the default shared-resource model for non-automotive virtualization.

In this model:

- Multiple guests can each have an assigned access window.
- Each access window targets the same underlying GPU execution domain.
- Only 1 guest can connect to the execution domain at a time.
- Ownership of the execution domain moves using arbitration. Arbitration involves yield, reset, and reopen behavior.

For the system, this is the simplest way to virtualize 1 GPU across multiple software domains, without requiring physically separated GPU hardware for each guest.

Benefits of a single-partition shared model

The benefits of the single-partition shared model are:

- Broad deployment flexibility
- Good hardware utilization
- A clean control model using access windows

What the guest sees when using a single-partition model

Each guest has its own stable, GPU-facing control interface. This stable interface makes this model intuitive to guest software, even though guests share the physical execution domain.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

However, guests do not run concurrently on the same execution hardware. They take turns owning it.

When the single-partition shared model is the best option

Single-partition shared systems are usually the best option when:

- Hardware utilization is more important than hard isolation.
- The platform requires 1 broadly reusable virtualization model.
- The product does not justify hardware-separated GPU execution domains for each software domain.

  This situation matters particularly in non-automotive deployments. The shared single-partition model is the most widely applicable model. However, systems that use this model often have limited control of the GPU workload mix. When arbitrary or poorly behaved content can run on the GPU, contention becomes less predictable, scheduling becomes harder to tune, and performance behavior becomes less controllable than in a tightly managed environment.

3.1.1 Performance and interference in the shared model

The shared single-partition model always involves contention risk because multiple guests reuse the same execution domain over time.

You can still tune the platform effectively. Effective arbitration policy can:

- Improve the average latency for selected guests
- Reduce visible interference in common workloads
- Bias the system toward the behavior you require from your product

Arbitration policy can improve workload behavior or favor 1 guest.

Policy cannot create a hard isolation guarantee in a shared execution domain.

Hardware separation provides the best isolation between guests. This is because it prevents the GPU execution for one guest from interfering with the execution for another. A time-sliced GPU cannot provide this kind of isolation or satisfy requirements that depend on it.

3.2 Multi-partition systems

Multi-partition systems use the isolation-oriented model that binds different guest domains to different hardware execution domains.

The multi-partition model enables concurrency and freedom from interference between guests. In a multi-partition design, guests can run at the same time on separate GPU execution domains. Physical separation between domains enables this concurrency.

This model uses:

- True physical separation

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- Software-visible partition assignment
- Shared system-level components that GPU partitioning does not affect

When using this model:

- The system no longer only decides which guest runs next. It also decides which guest owns each hardware-isolated execution domain.
- The design emphasis shifts from reuse and fairness toward isolation and predictability.

What this model guarantees

At the GPU execution-domain level, multi-partition systems provide a stronger separation boundary than time slicing. That is why this model is a good option when your platform requires hard GPU-side isolation between software domains. This kind of isolation between software domains can help you meet functional safety requirements at the system level.

Consider hardware partitioning during your wider system-level design. Hardware partitioning does not automatically make the whole SoC contention-free. CPU time, memory bandwidth, interrupt routing, and other shared system resources can still affect end-to-end behavior.

3.3 Hybrid systems

Hybrid systems combine hardware separation and time-sliced sharing.

The flexibility of the Access Manager means that you do not have to choose between partitioning and time slicing. In practice, partitioning and time slicing answer different questions:

- Partitioning isolates execution resources from each other.
- Time slicing decides how software shares 1 execution domain over time.

Your system can combine them in several ways.

Common hybrid patterns

Some example hybrid system setups that you can use are:

- One partition reserved for a single high-priority guest. For example, a safety critical application. Meanwhile, several lower-priority guests can share another partition over time, like Android or IVI.
- Separate partitions that different software domains use, with time slicing still in use inside 1 or more of those partitions.
- A system where some guests receive dedicated hardware execution domains and others share a different execution domain using arbitration.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

3.4 System considerations

During system design, you must consider several aspects of the virtualization.

Arbiter placement

You must decide where to place the arbiter in your platform.

In the reference software stack we provide, the arbiter runs as a kernel module in the host domain or in a dedicated arbitration virtual machine. Other placements are possible, including more privileged environments such as the hypervisor or a safety island. However, these alternative options might increase certification and integration costs.

Workload assumptions affect performance

A strategy that works well in a tightly controlled ecosystem might not translate directly to a broad ecosystem, for example Android.

When you know the workload in advance, tuning a shared model can be practical and effective. However, when arbitrary content can run on the GPU, behavior under contention becomes much harder to guarantee.

Access-window ownership planning

Whichever model you choose, the platform must have a coherent ownership plan for access windows.

At a system level, you must decide:

- Which guests own each access window or windows
- Which privileged software component controls access window assignments
- How interrupt routing follows the same ownership model
- How stage-2 mappings align with GPU control paths that are visible to guests

These decisions are part of the system architecture and affect the integration complexity and security.

3.4.1 Scheduling policy

The reference scheduler provides a baseline scheduling policy, not a finished product policy.

When you define the scheduling policy for your platform, you must consider:

- Guest priority
- Display or vsync-related timing
- Guest-specific latency and throughput targets
- Safety requirements

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

3.5 What to read next

The following chapters build on this virtualization approach guidance:

- System Architecture explains the mechanisms that make these virtualization models possible
- System Integration explains how to assemble the virtualization mechanisms into a working platform

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4. System Architecture

This chapter describes the system-level architecture that makes GPU virtualization work in practice. The key mechanism is the Access Manager, which sits between guest-visible control interfaces and the underlying GPU execution resources.

At a high level, the architecture has three layers:

- The GPU execution hardware that runs work for the currently selected guest
- The Access Manager, which controls register access, interrupts, resets, and ownership transfer
- The surrounding platform components, such as the hypervisor, system MMU, interrupt controller, and any secure or system-control software.

4.1 Hardware overview

The Access Manager is part of the GPU. It controls access to control registers and interrupts for the rest of the GPU, and it also participates in reset, clock, and virtualization flows.

For virtualization, the Access Manager provides a set of access windows. Each access window is a guest-visible control aperture that can be assigned to one Virtual Machine (VM). Platform software can keep that access window mapped to the same guest over time, while the Access Manager controls whether the window is actively connected to a GPU subinstance.

This is an important architectural property. A change of GPU ownership does not require the platform to rebuild the entire guest-facing GPU control map each time. Instead, the platform keeps a stable guest-visible interface and uses the Access Manager to control which window is open.

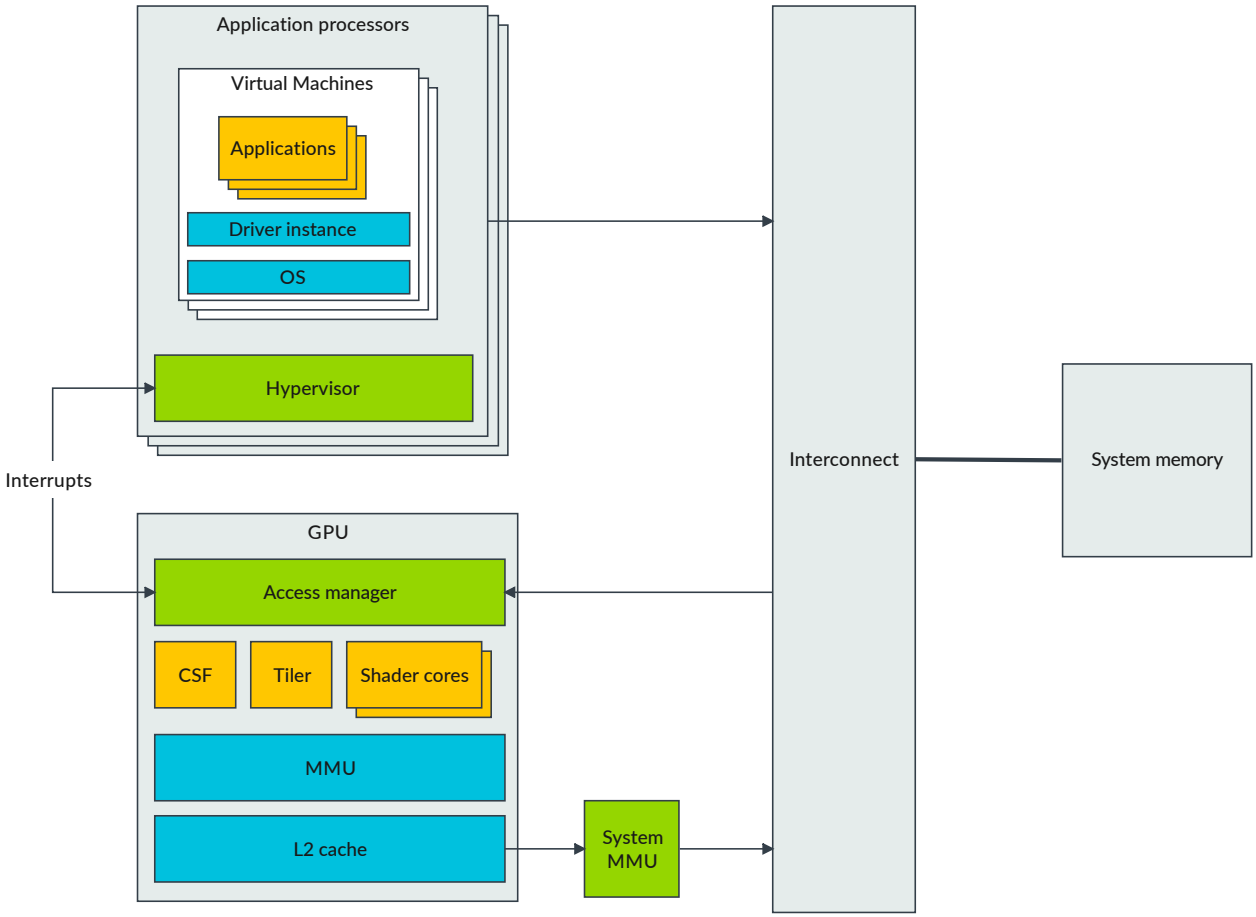
The GPU address map reflects this split:

- The first 1 MiB is reserved for Access Manager control registers.
- The remaining address space is used for access windows.
- The privileged control pages are arranged as 64 KiB regions.

In a shared single-partition implementation, all access windows target the same GPU subinstance, so only one window can be open at a time. In partition-capable AE implementations, access windows are associated with partitions, which allows multiple hardware-isolated execution domains to exist at the same time.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 4-1: Hardware overview



4.2 Software stack

At system level, the software stack can be described in four roles:

Guest driver instance
: The GPU kernel driver running inside a VM.

Firmware
: The software running on the GPU inside the active GPU subinstance.

Arbiter
: The software component that decides which VM should own a GPU subinstance next.

Hypervisor or equivalent privileged platform software
: The component that owns stage-2 mappings, access-window assignment, and the overall security model.

The exact deployment of these roles is platform specific, but the ownership boundaries should remain consistent.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Each guest typically has its own driver instance and its own assigned access window. The firmware belongs to the currently active execution environment and participates in yield and restart. The arbiter manages ownership changes. The hypervisor or equivalent privileged software remains responsible for system isolation and assignment policy.

In practice, these roles are usually implemented as a small set of cooperating software modules rather than as one monolithic virtualization component. The exact module boundaries vary by platform, but the functional split is consistent.

On the guest side, the kernel driver owns the guest-visible GPU programming model. It is responsible for access-window use, queue management, interrupt handling, and guest-visible recovery behavior. The firmware complements this by managing quiesce, save, restore, and restart behavior inside the active GPU subinstance.

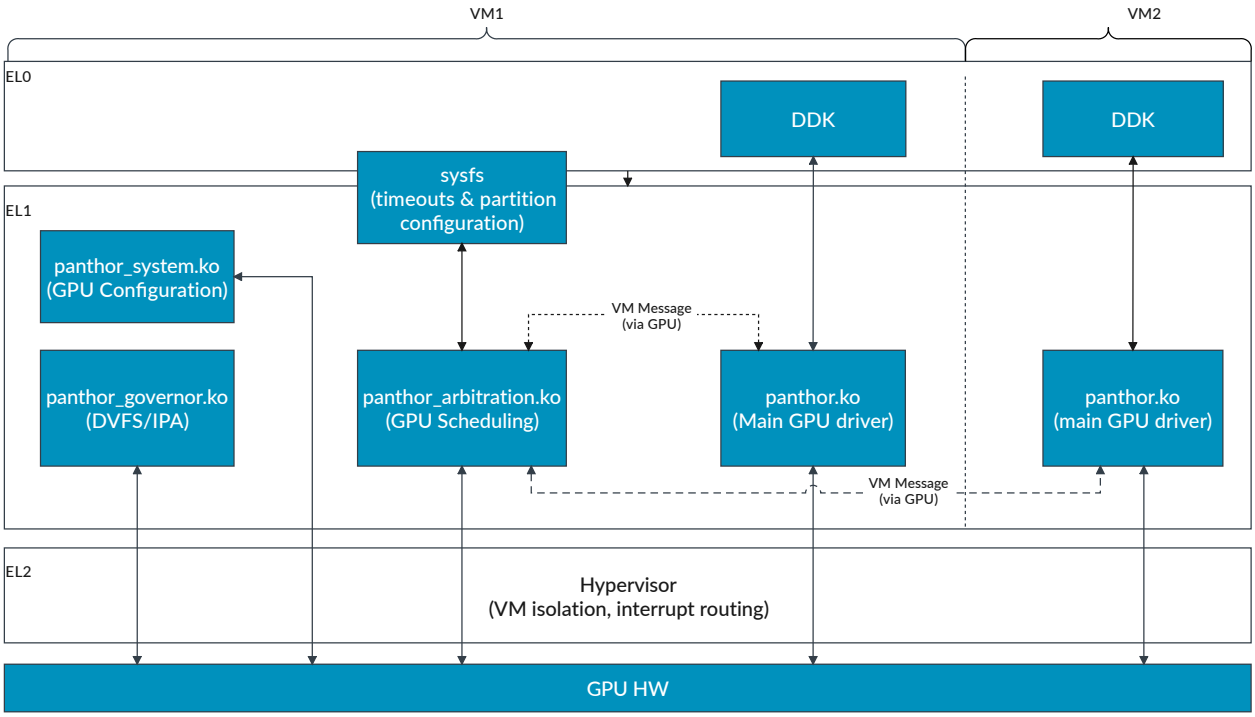
Outside the guest, the arbiter provides the scheduling and handover control needed to move ownership between guests. The hypervisor or equivalent privileged platform software remains responsible for assignment and isolation, including CPU stage-2 mappings, system-MMU configuration, interrupt routing, and overall security ownership.

The important point for the architecture is the contract between these roles, not the exact public name of one software module. Guest software owns guest-visible GPU operation, firmware owns execution-state handover, the arbiter owns scheduling policy, and privileged platform software owns isolation and assignment.

For more information about software-module detail, see the *Arm® Mali™ Immortalis™ GPU v10 Architecture and Later DDK Integration Manual*.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 4-2: Modules in the reference software stack for hardware virtualization



4.3 Access Manager

The Access Manager is the hardware component that enables the virtualization model. The Access Manager enables the platform to present each guest with a stable control interface while retaining hardware control over which guest owns the GPU execution resources.

This mechanism reduces the frequency of remapping and interrupt reconfiguration in the wider platform. With the Access Manager, those guest-facing interfaces can remain stable while hardware-controlled window open and close operations handle ownership changes.

4.3.1 Control pages and ownership boundaries

The privileged part of the Access Manager is split into control pages with different expected owners.

In the shared single-partition architecture, the main roles are:

- `SYSTEM` for privileged system control
- `GOVERNOR` for system-level power management
- `PARTITION_CONTROL` for access-window control
- `RESOURCE_GROUP` for arbiter-to-guest messaging

Partition-capable AE implementations extend this model with additional control pages for resource assignment and resource-group configuration. The architectural point is the same in both cases:

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

different parts of the control surface are intentionally separated so that the platform can map only the required functions to each privileged software component.

This separation is what allows the arbiter to exist outside the hypervisor when that is the preferred system design. The hypervisor still controls assignment and memory translation, while the arbiter can be given access only to the control pages needed for scheduling and messaging.

4.3.2 Access windows in the Access Manager

An access window is the guest-facing interface to one GPU control path. The intended model is that each access window is assigned to one VM on a semi-permanent basis.

It is useful to separate the two directions of access involved in virtualization:

AXI-subordinate
: CPU-initiated access into the GPU register space through an access window.

AXI-manager
: GPU-initiated access out to system memory while that access window is active.

The Access Manager supports up to 16 access windows. For the AXI-subordinate path, the hypervisor controls access by mapping only the assigned access-window register block into the guest's permitted memory range. The platform must align four things consistently:

- CPU stage-2 mappings, so each guest can access only its assigned windows.
- GPU system-MMU mappings, so GPU memory accesses from that window can reach only the correct guest memory.
- Interrupt routing, so each access-window interrupt reaches the correct guest.
- Arbiter policy, so window ownership changes follow the intended scheduling model.

When an access window is open, the driver instance using that window can access the GPU subinstance through that path. When the window is closed, the guest retains access only to the window-control portion of the interface. Attempts to use the rest of the window while it is closed are treated as invalid access.

Closing an access window does not mean all guest-side GPU software activity stops. The guest can no longer directly access the GPU subinstance control interface, but queue contents, firmware state, and other driver-managed state remain in system memory. In a CSF-based system, user space can still append commands to an existing queue in that memory while the window is closed. Those commands are not consumed until the GPU subinstance is made available to that guest again.

4.3.3 Arbiter and hypervisor split

The Access Manager does not replace the hypervisor. It reduces the amount of work required for normal ownership transfer, but it does not provide a full virtualization solution by itself.

The division of responsibilities is:

- The hypervisor controls stage-2 memory translation and maps access-windows into the address space for each guest.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- The arbiter decides which guest should own a GPU subinstance next.
- The guest driver manages the currently active guest execution environment.
- The Access Manager enforces the hardware transition between those states.

The architecture is designed so that the arbiter can run either as part of the hypervisor or as a separate privileged software component, including in a dedicated VM. If it is separate, it is expected to use the scheduling and messaging pages exposed by the Access Manager, while the hypervisor retains control of system isolation.

4.3.4 Interrupt and message paths

The Access Manager also sits in the interrupt path.

In the shared single-partition architecture, the external interrupt model separates:

- System-wide interrupts for privileged platform handling
- Control-path interrupts for arbiter-side management of a GPU subinstance
- One guest-facing interrupt line per access window.

For each access window, the guest-facing interrupt behavior depends on whether the window is open. When the window is open, window-control interrupts and GPU-subinstance interrupts are combined onto that window's interrupt line. When the window is closed, only the window-control interrupts remain visible through that line.

The Access Manager also provides a bidirectional message path between each driver instance and the arbiter. This messaging path is not the whole scheduling mechanism, but it provides a defined control channel for coordination between the guest-side driver and the arbiter-side software.

In partition-capable AE implementations, the interrupt structure becomes more granular because partitions and resource groups are independently visible control domains. The system-integration requirement is unchanged: interrupt routing must match ownership.

In these systems, the arbiter handshake can also communicate the maximum core mask associated with a guest's resource group. This should be treated as the upper bound of the resource that can be exposed to that guest. On any given resume, the guest can be granted only a subset of that mask, and the granted subset can change across suspend and resume cycles.

4.3.5 Cooperative handover

Ownership transfer is a cooperative handover, not a transparent background switch.

In the preferred design, the arbiter uses an asynchronous yield path built around `YIELD_NOW`, `YIELD_IDLE`, and `YIELD_DONE`, rather than relying only on a host-driven stop path. In this path, firmware can complete the GPU-side yield and notify the Access Manager asynchronously in hardware. This decouples yield completion from host CPU scheduling latency, which reduces the risk of timeout or `GPU_LOST` behavior caused only by delayed CPU scheduling, even though the host still participates in the wider suspend, handover, and resume flow.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The handover sequence is:

1. The arbiter decides that another guest should receive the GPU subinstance.
2. The arbiter issues either `YIELD_NOW` or `YIELD_IDLE`, depending on how urgently the switch is needed.
3. The firmware running in the current execution environment quiesces work, saves the required state, and signals `YIELD_DONE`.
4. The Access Manager closes the current access window and resets the GPU subinstance.
5. When reset completes, the arbiter selects the next access window and issues `OPEN_WINDOW`.
6. The Access Manager prepares the new window, clears selected state-holding RAMs, opens the window, and signals that the new owner can proceed.
7. The new driver instance powers up the required GPU blocks and boots or resumes the firmware.

This sequence is central to the isolation model. The reset and RAM-clearing stages are part of the architectural boundary between one guest's execution environment and the next.

It is also important to distinguish between the GPU subinstance and the software-visible queue state stored in system memory. While a guest is not the active owner, its existing CSF queues can still be updated in memory by software running in that guest. The queued commands are then consumed when that guest's time-slice resumes and the GPU subinstance continues processing from the restored state.

4.3.6 Why reset is part of the architecture

Reset is not an implementation detail that can be ignored in software documentation. It is part of how the Access Manager prevents transient state from one guest from remaining visible when another guest takes ownership.

This has direct system-level consequences:

- Ownership transfer has non-zero latency
- Firmware participation is required for orderly handover
- A guest that does not yield promptly can force timeout and recovery behavior
- A forced close can be visible to software as loss of work or device-lost style failure.

The architecture also supports guest-initiated reset of the active GPU subinstance without changing which access window is assigned. Separately, the platform can fully reset an access window before assigning it to a different guest, which clears persistent guest-specific control state in that window, for example when re-assigning an access window to another guest.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

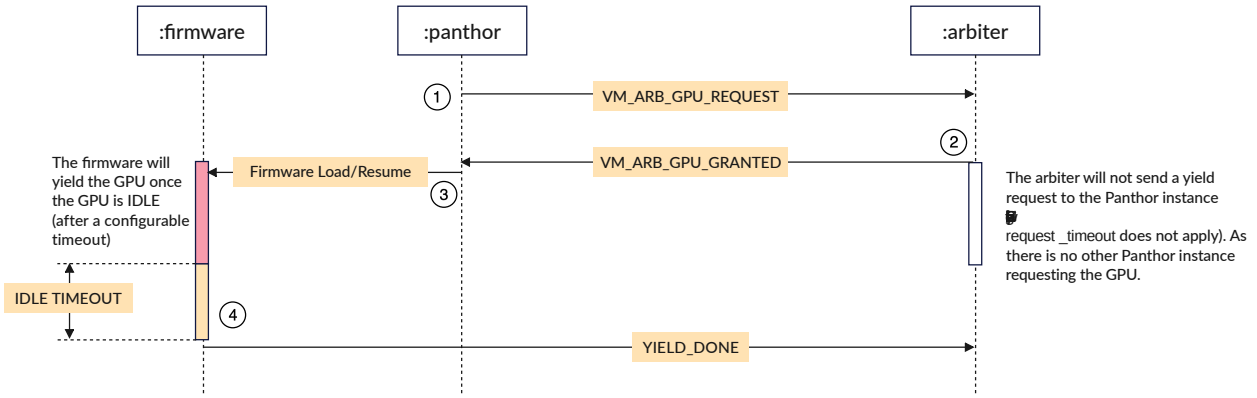
4.3.7 Time-slicing examples

The following examples illustrate common time-sliced behaviors.

No Yield (single VM)

The following example shows the degenerate baseline. One guest owns the GPU execution domain and no inter-guest arbitration is needed.

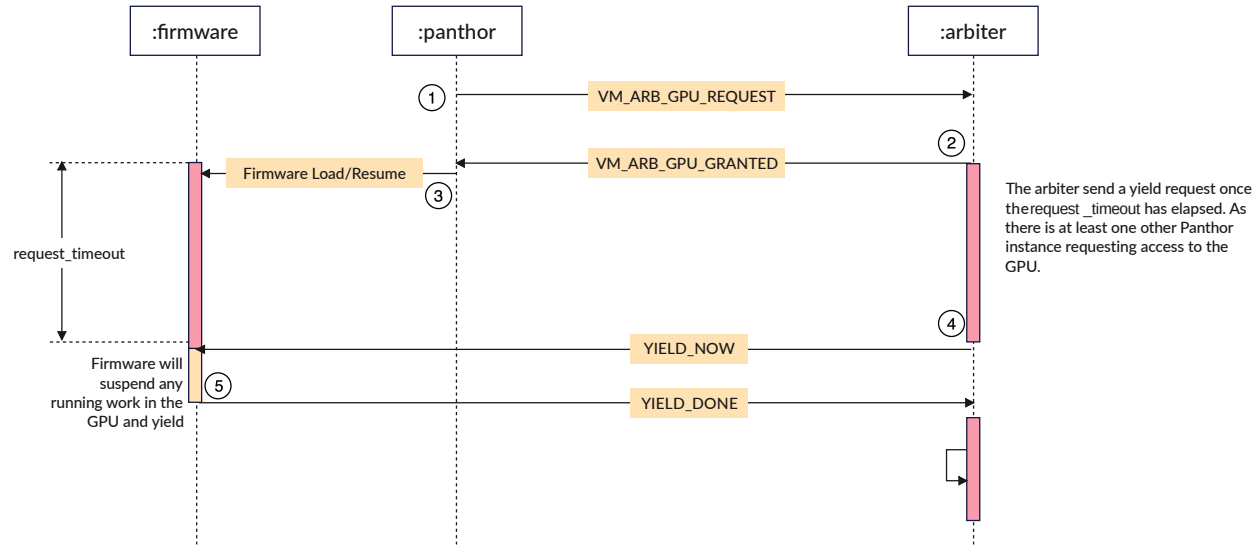
Figure 4-3: No Yield



Early Yield (yield on idle)

The following example shows the low-disruption path in which ownership changes only after the current guest reaches an idle point.

Figure 4-4: Early Yield

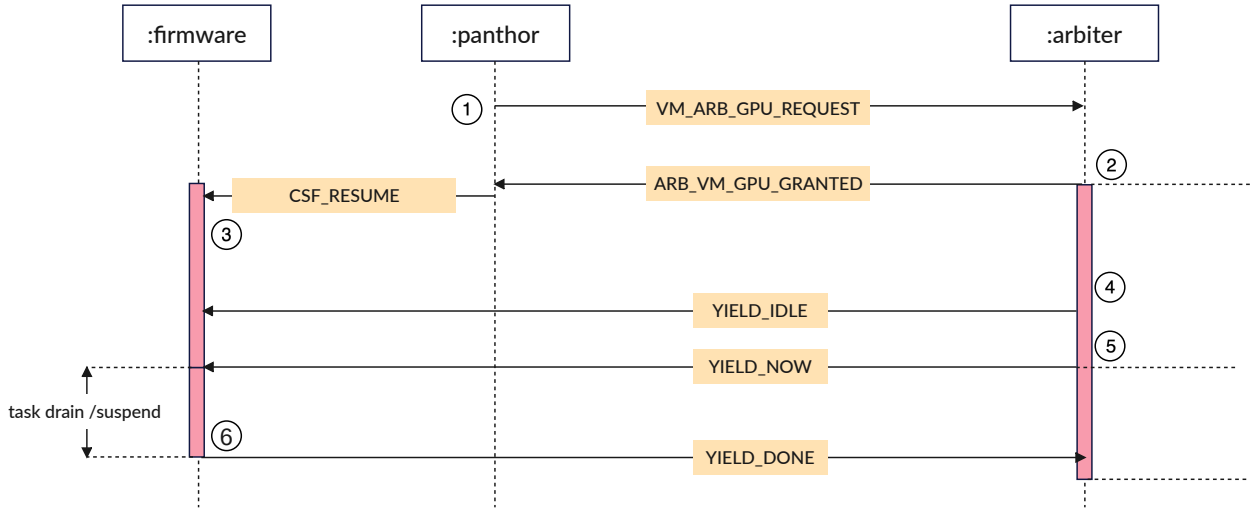


Normal Yield (multiple VMs)

The following example shows the standard shared model. One guest is asked to stop, it yields in an orderly way, and ownership transfers to the next guest.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

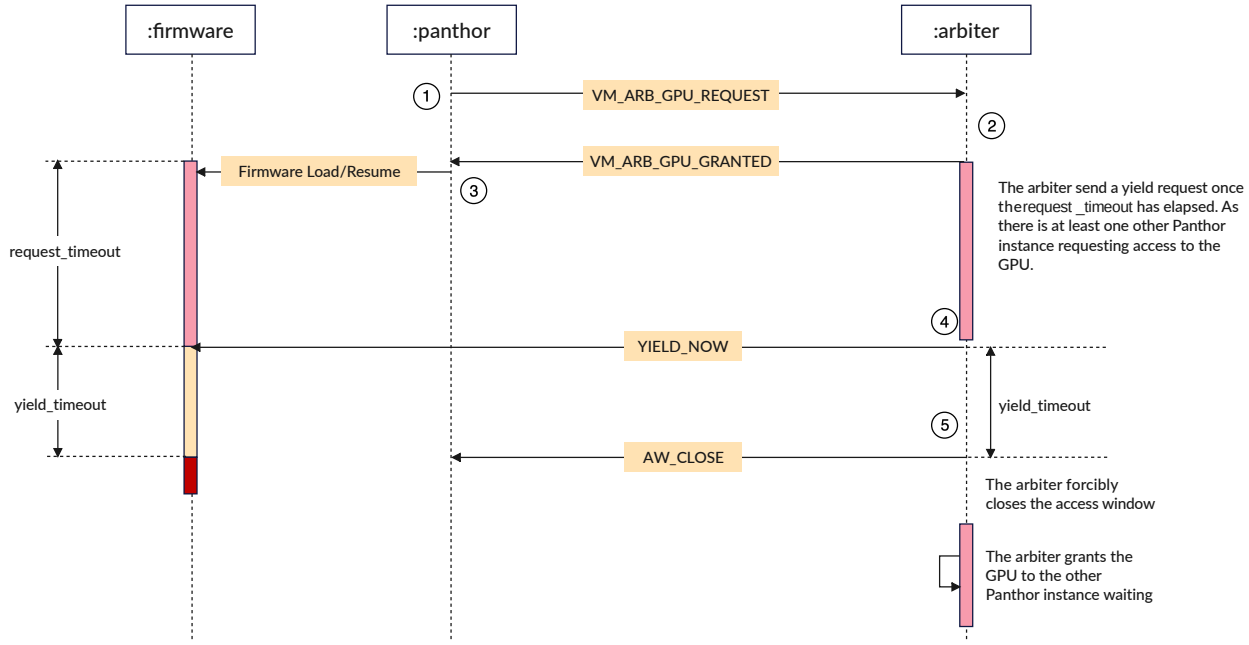
Figure 4-5: Normal Yield



Failure to Yield (`GPU_LOST`)

The following example makes the recovery path visible. Time-sliced sharing includes timeout handling and the possibility of lost work when orderly handover does not complete.

Figure 4-6: Yield Failed

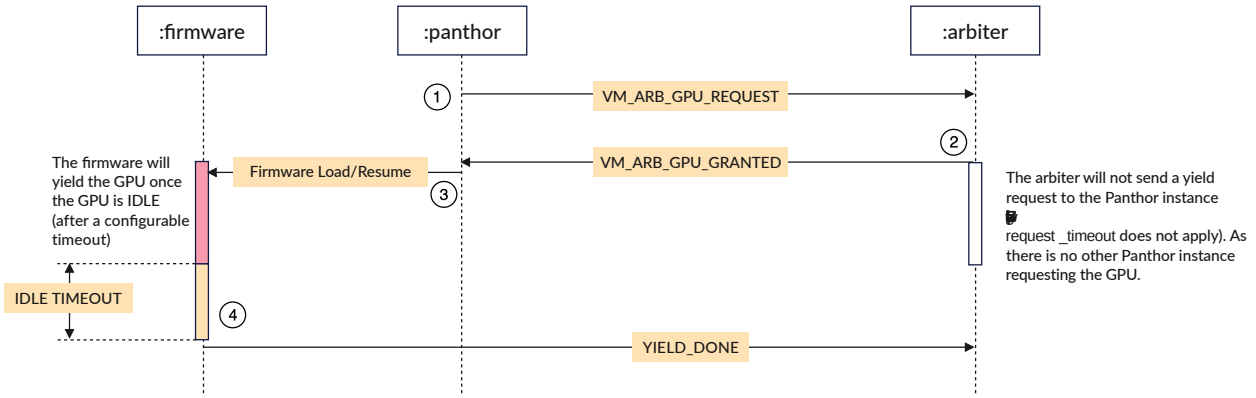


Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.3.7.1 Self yield use case

This example shows the degenerate baseline in which one guest owns the GPU execution domain.

Figure 4-7: No Yield

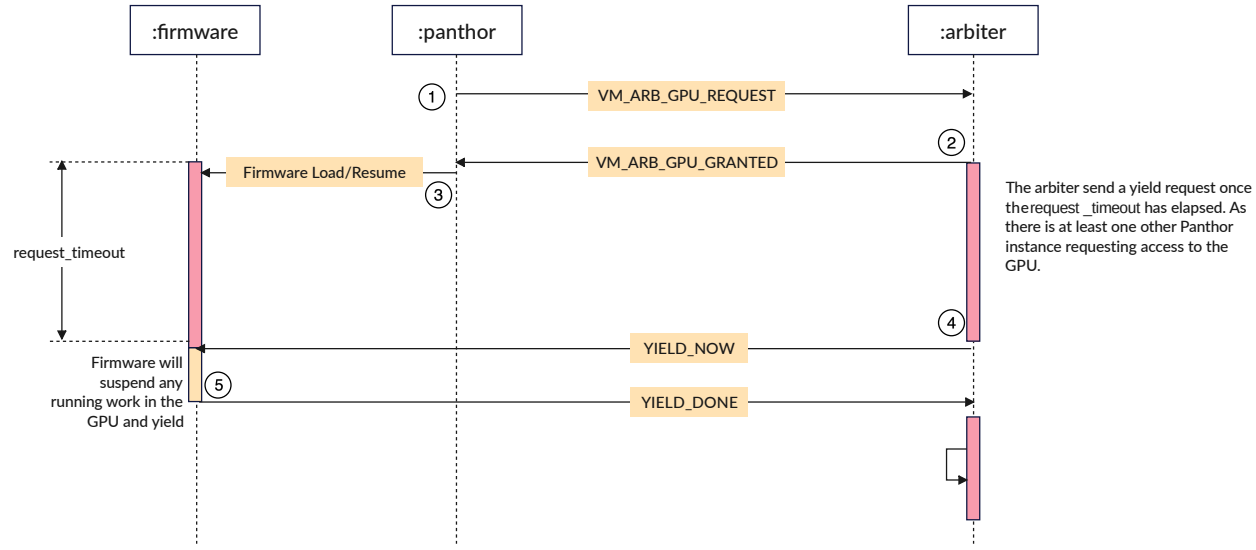


This is the degenerate baseline. One guest owns the GPU execution domain and no inter-guest arbitration is needed.

4.3.7.2 Multiple requesters time slicing use-case : Successful yield

This example shows the low-disruption path in which ownership changes only after the current guest reaches an idle point.

Figure 4-8: Early Yield



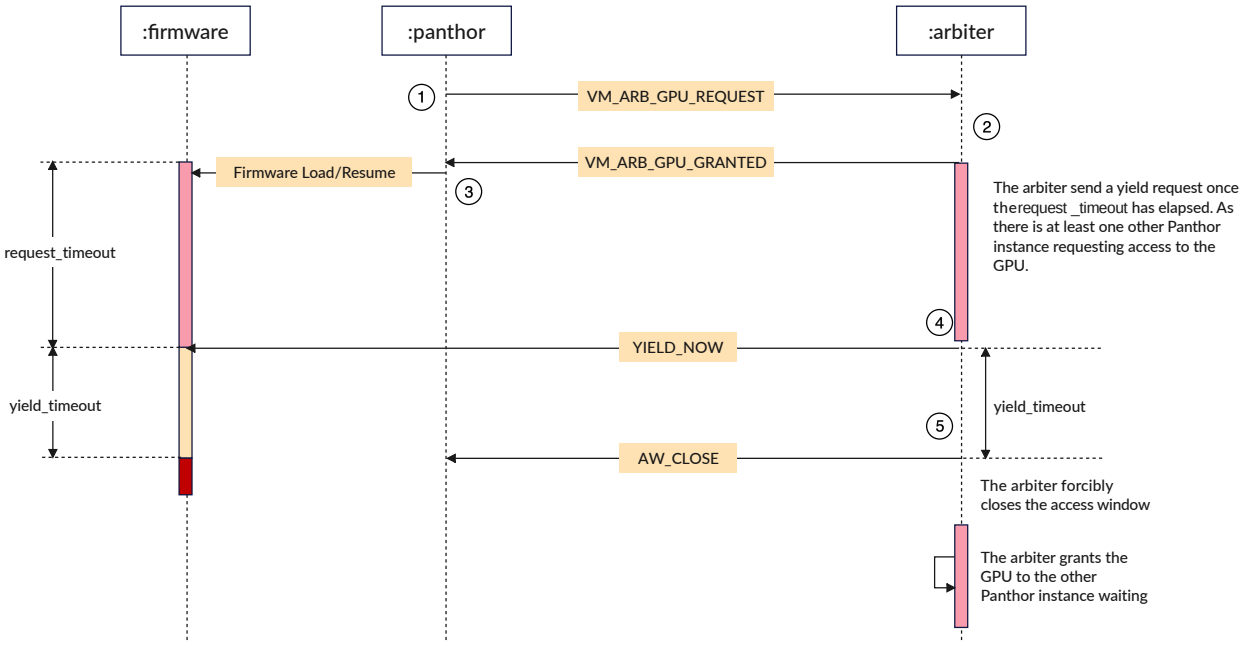
This is the low-disruption path in which ownership changes only after the current guest reaches an idle point.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.3.7.3 Failure to Yield (GPU_LOST)

This example makes the recovery path visible when orderly handover does not complete.

Figure 4-9: Yield Failed



This example makes the recovery path visible. Time-sliced sharing includes timeout handling and the possibility of lost work when orderly handover does not complete.

4.3.8 External memory isolation

The Access Manager is necessary, but not sufficient by itself, for a secure virtualized design.

The GPU does not provide stage-2 translation. Consider the implications separately for each access direction:

- On the AXI-subordinate path, the hypervisor controls which access window register ranges are mapped into each guest.
- On the AXI-manager path, system memory isolation depends on an external system Memory Management Unit (MMU), Memory Protection Unit (MPU), or TrustZone based protection mechanism, depending on the design goals.

For AXI-manager accesses, the GPU exposes a stream identifier so the surrounding System on Chip (SoC) can apply the correct translation and protection rules. Each access window can generate two StreamID values, one for normal operation and one for protected mode.

The exposed `AxMMUSID` includes:

- Bits `[4:1]`: the currently active access window index
- Bit `[0]`: the GPU protected state indication

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

When the GPU enters protected mode, `AxMMUSID[0]` is set. This gives one effective StreamID for the normal case and a second effective StreamID for protected mode accesses from the same access window.

The hypervisor must configure the system MMU so that the StreamIDs associated with an access window are permitted to access only the memory owned by the corresponding guest. If a system needs additional StreamID meaning beyond the access window index and protected state indication, that encoding must be added outside the GPU.

The key integration requirement is that guest-visible windows, interrupt routing, and the memory isolation policy must be consistent with the same ownership model.

4.3.9 Shared and partition-capable systems

The Access Manager framework supports both shared and partition-capable products.

In a shared single-partition system, multiple guests share one GPU subinstance over time through time slicing. In a partition-capable AE system, access windows map to partitions, so different guests can run concurrently on different hardware-isolated execution domains.

The same architectural principles still apply:

- Each guest is given a defined control interface
- Privileged software controls ownership and assignment
- Interrupt routing and memory isolation must match that ownership model
- Time slicing can still be used within a partition when required.

This is why partitioning and time slicing should be treated as complementary mechanisms. Partitioning defines the hardware isolation boundary. Time slicing defines how one execution domain is shared over time.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5. System Integration

This chapter describes how to assemble the hardware and software elements of the virtualization architecture into a working system. It focuses on ownership boundaries, software responsibilities, and the order in which platform integration usually happens.

The software module split varies by product and operating environment. The integration principles stay the same, each guest needs a well-defined control path, privileged software must preserve isolation, and scheduling must remain consistent across normal execution, yield, and recovery.

5.1 General integration flow

Platform integration normally starts with the target virtualization model. The system designer must first decide whether the product uses a shared single-partition model, a partition-capable model, or a hybrid arrangement. That decision affects how access windows are assigned, how interrupts are routed, and where arbitration policy must be enforced.

Bring-up is usually easiest when it starts in the simplest software environment and adds platform complexity in stages.

Once the model is chosen, the platform can be integrated in a predictable order:

1. Bring up a single OS environment with multiple access windows.
2. Assign ownership of access windows, interrupts, and privileged control interfaces.
3. Bring up the guest driver, firmware, guest-visible interrupt paths, and basic arbitration flow.
4. Validate ownership transfer, normal yield, idle yield, timeout recovery, and reset behavior in that simpler environment.
5. Add the production VM split, stage-2 mappings, and broader platform integration.
6. Add runtime power-management coordination, then tune timeout and policy behavior for the target workload.

This sequence keeps the isolation model clear. The guest-visible interface, the privileged ownership model, and the memory-protection configuration should all reflect the same system design.

5.2 Software component responsibilities

The software stack is best understood as a set of cooperating components with distinct responsibilities.

The guest driver owns the guest-visible GPU programming model and is equivalent to the non-virtualized GPU driver.

The guest driver is responsible for the following:

- GPU initialization within the guest

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- Queue setup
- Memory allocation
- Stage-1 GPU mappings
- Interrupt handling
- Fault handling

Virtualization-specific additions center on the access-window interface, which combines interrupts and bidirectional messages to coordinate arbitration events between the guest and the arbiter.

Much of the driver-managed state remains in system memory. Queue contents, address-space state, and firmware-related data structures can remain valid even while the guest is not the active owner of the GPU subinstance. As a result, user space can continue to submit work to existing queues, but that work is not consumed until the access window is reopened for that guest.

The firmware owns execution-side handover inside the active GPU subinstance. On suspend, it stops submitting new work to the GPU engines and waits for outstanding work to complete. Firmware performs this handling per CSG, so only application contexts that fail to complete a yield in time lose work in the timeout case. In the normal case, a yield is intended to behave like a suspend rather than a destructive reset.

At the software level, the two main yield commands have distinct meanings.

`YIELD_NOW`
: The current owner should stop as quickly as possible.

`YIELD_IDLE`
: The current owner should yield when its current queue of work has drained.

The second case is intentionally asynchronous, so the software stack needs a clean separation between arbitration events and the guest driver's normal queue-management path.

When ownership returns to a guest, the execution environment is resumed from system memory rather than rebuilt from scratch. Firmware state is restored into tightly coupled memory and execution resumes from the preserved software state. This warm-boot path is much faster than a cold boot and is one of the key reasons the time-sliced model is practical.

Arbiter

The arbiter owns scheduling policy and handover control, its responsibilities are:

- Deciding which AWs are enabled at a time.
- Issuing yield requests.
- Selecting the next access window to open.
- Handling timeout and recovery decisions, if orderly handover does not complete.

The baseline reference scheduler uses a simple fair round-robin policy, but the software structure is intended to allow you to replace or extend that scheduling policy for your own systems.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Hypervisor

The hypervisor, or equivalent privileged platform software, owns assignment and isolation and performs the following actions:

- Controlling CPU stage-2 mappings for guest access to access windows.
- Deciding which guest should own a GPU subinstance next.
- Configuring the system MMU so GPU memory accesses are constrained to the correct guest memory.
- Routing interrupts correctly.
- Retaining ownership of privileged security and assignment policy.

The architectural split described in this guide is not specific to a single hypervisor implementation.



Note

These role boundaries are more important than the exact module names used in one software stack. A platform can move functionality between components, but it must preserve clear ownership of guest programming, execution-state handover, scheduling, and isolation.

5.3 Reference stack and partner adaptation

The reference software stack demonstrates a complete and working architecture, but it should not be treated as the only production software arrangement.

Some parts of the stack express architectural requirements, for example:

- Access-window ownership.
- The yield and handover sequence.
- The separation between guest control, arbitration, and privileged isolation.

Those boundaries should remain stable even if the software is restructured for a specific product.

Other parts of the stack express baseline implementation policy, for example:

- Where the arbiter is placed.
- How the default scheduler behaves.
- How platform-specific policy decisions are implemented.

These areas are expected to vary between products.

In the current Linux-based implementation, the responsibilities are split across separate components for the guest driver, governor or power-management policy, privileged system control, and arbitration.

The code structure also makes the virtualization support more modular than extending a single power-management path in the main driver. In particular, the arbitration side is implemented as

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

a separate virtualization component with its own partition-control and resource-group handling, while the guest-side driver interacts with it through the access-window protocol.

From a Linux device-model point of view, this is best understood as a parent-child platform-device relationship rather than as a single monolithic driver. The arbitration component owns the top-level virtualization device and creates child devices for the subsystem interfaces it manages. That structure helps separate system-control responsibilities of:

- System control
- Arbitration
- Guest-driver

In a way that reflects the hardware ownership model.

Yous will often need to adapt the reference stack for your own system. Typical changes include:

- Moving the arbiter to the platform component that best matches the system architecture.
- Replacing or extending the baseline scheduler with product-specific policy.
- Integrating the privileged control model with the platform's existing security and virtualization software.
- Tuning timeout, recovery, and ownership-transfer behavior for the expected workload mix.

The goal of the reference stack is to provide a correct architectural baseline. It is not intended to replace platform-specific integration work.

5.4 Bring up the software

Follow the same ownership model during bring up that you plan to use on the final platform.

About this task

For early bring up, you must start with multiple access windows in one Operating System (OS) before you split components across Virtual Machines (VMs). This simpler setup allows you validate the access-window protocol, arbitration flow, switching behavior, and key driver and firmware interactions first.



Early bring up helps you find most functional issues before you add hypervisor integration, cross-VM power management, and workload-specific timeout tuning.

The following procedure shows how to bring up software.

Procedure

1. Configure address-space ownership.
    - CPU stage-2 mappings must ensure each guest can access only its assigned access-window register ranges.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- Configure the system Memory Management Unit (MMU) so GPU memory accesses from an active access window can reach only memory owned by the corresponding guest.

2. Establish privileged control ownership.
   - Assign access windows, map privileged control pages only to the correct software components, and define which component owns scheduling, reset, recovery, and power-management policy.
3. Bring up the guest driver and guest-visible interrupt routing.
   - The active execution environment can then boot firmware and enter normal operation.
   - In the early single-OS phase, validate handshake behavior, GPU-request flow, access-window open and close behavior, orderly yield, and queue persistence across time slices.
4. Enable the arbiter and validate the ownership-transfer path.
   - Cover normal switching, yield on idle, timeout behavior, forced close and recovery, and any resulting guest-visible error handling.
5. Split components across multiple VMs only after this baseline is stable. At that point, the main new concerns are stage-2 mappings, interrupt routing, ownership boundaries between VMs, and cross-VM runtime power-management ordering.
   - Document hypervisor-specific configuration details, see pKVM worked example, because the ownership principles stay the same even when configuration mechanisms differ.

Next steps

Tune the platform, and stop after timeout and scheduling-policy tuning for the expected workload mix. This next step includes runtime power-management coordination and any System Control Processor (SCP) or System Control and Management Interface (SCMI) integration.

5.5 Queue and state lifecycle

Virtualization depends on understanding which state is held in the GPU subinstance and which state remains in system memory.

Queue state, firmware-managed state, and other driver-visible data structures can remain in system memory even when a guest is not the active owner of a GPU subinstance. In a CSF-based system, user space can continue appending commands to an existing queue in memory while the corresponding access window is closed. Those commands are not consumed until that guest becomes the active owner again.

This means that not currently owning the GPU subinstance is not the same as all GPU-related software state is inactive. Integration work must account for the difference between direct hardware access, which is controlled by the Access Manager, and software-visible queue and state management, which continues in system memory.

The reset and handover path must also be handled carefully. Some state is intentionally cleared as part of ownership transfer so the next guest does not observe residual state from the previous owner. Other state remains guest-owned in system memory and is used when execution resumes. Recovery behavior must preserve this distinction, because failures in yield or reset handling can appear to guest software as lost work or device-lost-style behavior.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The normal yield path is therefore a suspend and warm-resume cycle. The timeout path is different. If yield does not complete in time, the platform might need to force recovery, and the resulting behavior is closer to a device-lost condition for the affected work. Integration and validation must cover both paths explicitly.

5.6 Non-automotive systems

Many non-automotive systems use a shared single-partition deployment model. This keeps the hardware model simple, but it also requires the software stack to handle contention, scheduling policy, timeout behavior, and a wider range of workload variation.

These systems often place particular emphasis on the following areas:

- Clean integration with the chosen virtualization framework.
- Correct system-MMU ownership and memory protection.
- Robust arbitration and recovery behavior for mixed workloads.
- Scheduler policy that reflects product goals such as latency, throughput, or user experience.

Because the workload mix is often less tightly controlled than in safety-oriented deployments, tuning and validation are especially important.

5.7 Automotive systems

Automotive and other isolation-oriented systems place greater emphasis on explicit ownership, determinism, fault handling, and hardware separation.

When partitions are available, the integration flow must account for both partition assignment and time slicing within each partition. This adds platform decisions about which guests require hardware-isolated execution domains and which guests can share one execution domain over time.

These systems often place particular emphasis on the following areas:

- Explicit ownership of partitions, access windows, and privileged control paths.
- Well-defined arbitration and recovery behavior.
- Integration with the platform's safety and security architecture.
- Validation of predictable behavior under fault and recovery conditions.

The architectural model remains the same, but system goals usually place more weight on isolation and predictability than on maximizing shared hardware utilization.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

6. Power management model

This chapter explains how runtime power management, Dynamic Voltage and Frequency Scaling (DVFS), and dynamic core scaling fit into a virtualized Graphics Processing Unit (GPU) system.

The primary responsibilities of the software stack are to:

- Manage GPU power-state transitions.
- Coordinate state changes across software domains.
- Host performance-management policies.

Ownership transfer is a secondary consideration to these functional requirements.

6.1 Core message

In a non-virtualized system, runtime suspend and resume are often treated as a local driver concern. In a virtualized system, that is no longer sufficient. The guest driver still integrates with the Linux runtime power management framework. However, multiple software components can host the hardware and software resources that it depends on. In some systems, these resources also span multiple Virtual Machines (VMs).

As a result, power management is best addressed through coordinated decision-making across system components:

- The guest driver needs a simple `request power` model.
- The underlying power-on and power-off sequence can involve several dependent components.
- Cross-VM suspend and resume ordering must be preserved.
- Dynamic Voltage and Frequency Scaling (DVFS) and dynamic core scaling policy can be hosted in different places depending on the platform design.

6.2 Main software roles

At the system level, Power Management (PM) typically involves the following roles:

Guest driver
: Integrates with Linux runtime PM and requests power when it needs to use the Graphics Processing Unit (GPU).

Governor function
: Monitors GPU activity and drives Dynamic Voltage and Frequency Scaling (DVFS) or dynamic core scaling decisions.

Privileged system-control software
: Manages the platform-facing control path for power, clocks, and regulators.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Platform power-management mechanism

Coordinates ordering across software boundaries.

The governer control page is designed to support different governor implementations. The governor can be implemented in the host kernel, in a System Control Processor (SCP), or across both. For example, Intelligent Power Allocation (IPA)-style calculations can be performed in the host kernel, while voltage and clock control are performed by the SCP.

The guest driver remains independent of the governor implementation and placement. It interacts with runtime PM through the standard Linux runtime PM model, while the platform determines how power-management requests are implemented and where policy decisions are made.

6.3 Powered-off behavior

When the GPU is powered off, GPU registers are not accessible. Any attempt to access a GPU register results in a bus error. This restriction also applies to the access-window communication path. Therefore, a guest cannot communicate with the arbiter through its assigned access window while the GPU is powered off.

For the guest driver, the abstraction is simply to request power. How the platform powers the relevant hardware blocks, restores the communication path, and enables access through the access window is implementation-specific. For example, the system can use the System Control and Management Interface (SCMI) protocol through the Linux generic power domains `genpd` framework to request GPU power-up and restore the communication.

6.4 Runtime PM integration

The guest driver should integrate with Linux runtime PM in the same way that a normal kernel driver does. In a virtualized system, however, the driver is not necessarily controlling only one local hardware block. A runtime-resume request from an access-window driver can need to bring up a chain of dependent components before the access window becomes usable.

The ordering requirement can be expressed at role level as:

Resume

System control, then governor, then arbiter-side components, then the guest driver.

Suspend

The reverse order, starting from the guest driver and unwinding back toward system control.

Within a Virtual Machine (VM), Linux can enforce this ordering by using standard mechanisms such as device links. Across VM boundaries, the platform requires an implementation-independent coordination mechanism to enforce the same dependency without relying on a particular hypervisor design.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

6.5 An Arm-style cross-VM approach

One suitable Arm-style approach is to use the System Control and Management Interface (SCMI) power domains integrated through Linux `genpd` and runtime PM. In this model, the guest-facing devices still participate in normal runtime PM, but the cross-VM ordering is coordinated through SCMI state changes and notifications.

In the reference Linux design, the power domains used for this purpose are best understood as logical signaling domains rather than as direct representations of physical power rails. Their purpose is to preserve ordering and reference counting across software boundaries. The actual implementation of clocks, regulators, or other physical controls remains platform-specific.

This distinction matters because some systems might choose a different backend. For example, you can model the dependencies through virtual clocks or regulators instead. For that reason, this guide describes SCMI plus `genpd` as a suitable implementation approach rather than as a hard architectural requirement.

6.6 Logical and physical power control

The software interfaces that coordinate runtime PM do not have to map directly to physical power domains.

In some designs, the coordination layer is entirely logical and exists only to preserve ordering between:

- System control
- Governor
- Arbiter-side components
- Access-window drivers

The physical control of power, clocks, and regulators can be implemented elsewhere, for example in an System Control Processor (SCP) or platform firmware. Therefore, this guide focuses on the behavior that is visible to the guest driver and on the required ordering, rather than assuming a fixed implementation across all System On Chips (SoCs).

6.7 Dynamic voltage and frequency scaling and dynamic core scaling

The governor role observes GPU activity and drives Dynamic Voltage and Frequency Scaling (DVFS) and dynamic core-scaling decisions. Different product architectures can host that role in different places. The role should still have enough visibility and control to make decisions without tightly coupling the rest of the virtualization stack to one implementation.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

6.8 Guest driver assumptions

The guest driver relies on a simple Power Management (PM) contract:

- Request power before accessing the GPU.
- Release power when the GPU is no longer required.
- Do not access GPU registers while the GPU is powered off.
- Rely on platform integration to make dependent components available in the required order.

The guest driver does not need to know whether the platform is using the System Control and Management Interface (SCMI), `genpd`, virtual regulators, an SCP-owned governor, or some other equivalent mechanism. Those choices belong to the integration layer.

The runtime PM contract described in this guide does not include firmware lifecycle management. The guest driver manages firmware boot, suspend, and warm-resume behavior. For information about these behaviors, see System Architecture.

6.9 System-integration implications

For system integration:

- You must peserve runtime PM ordering even when the relevant components are distributed across multiple Virtual Machines (VMs).
- When the platform powers off the GPU, access-window messaging becomes unavailable.
- Use the System Control and Management Interface (SCMI) with `genpd` to implement this ordering in a hypervisor-agnostic Linux system.
- Use alternate platform mechanisms if they preserve the same functional behavior.
- Governor placement and Dynamic Voltage and Frequency Scaling (DVFS) policy remain platform choices, provided the required control information is available.

For more information about yield, suspend after `WINDOW_CLOSED`, and ownership-transfer recovery as architectural behaviors of the virtualization model, see System Architecture.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

7. Hypervisor integration

This chapter explains how to integrate Graphics Processing Unit (GPU) virtualization into a hypervisor-managed system. It does not document a specific hypervisor configuration. Instead, it describes responsibilities that remain consistent across virtualization frameworks.

The exact configuration mechanism varies by hypervisor. Some systems describe these assignments in the device tree. Others use static platform configuration, or a combination of both. Those details are hypervisor-specific, but the architectural requirements are the same.

At this level, the key concerns are:

- Assignment of access windows to guests.
- CPU stage-2 mappings for guest-visible register access.
- System-MMU configuration for GPU memory access.
- Interrupt routing for access-window and privileged control paths.
- Ownership boundaries between guest software, arbiter-side software, and privileged platform software.

7.1 Hypervisor responsibilities

The hypervisor, or equivalent privileged software, is responsible for system isolation. That means it must:

- Map only the assigned access-window register range into each guest.
- Configure the system MMU so GPU memory accesses stay within the correct guest-owned memory.
- Route interrupts to the correct guest or privileged software component.
- Retain ownership of privileged control pages and assign them only to the software components intended to own them.
- Control guest access to protected memory when GPU protected mode is supported.

These responsibilities remain with the hypervisor even when the arbiter runs outside the hypervisor, for example, in a host kernel or a dedicated VM.

7.2 Access-window assignment

The normal software model assigns one access window to each guest. The access window provides the guest-visible register block and interrupt path for a time-sliced GPU resource or, in a partition-capable implementation, for a partition-backed execution domain.

The hypervisor assigns an access window to a guest for the lifetime of a guest. The hypervisor can reassign the access window to a different guest only after terminating the original guest.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Reassignment requires more than yielding the GPU subinstance. Before an access window is assigned to a different guest, the privileged access-window reset operation must be performed so guest-specific control state in that window is cleared. Resetting the GPU subinstance alone is not sufficient when the access window itself is being reprovisioned to another guest.

7.3 Privileged ownership model

Not all privileged GPU control pages carry the same security and system-integration implications.

SYSTEM
: Highly privileged

  It can affect system-wide GPU reset behavior and broader system interconnect behavior, so it must remain under tightly controlled privileged ownership.

GOVERNOR
: Privileged

  It exposes GPU-wide performance information and can therefore reflect activity outside the isolated execution domain of one guest.

The arbiter usually owns partition-control and resource-group control pages together because it requires both. These pages affect performance, scheduling, and fairness across multiple guests, but they do not provide direct access to guest data.

These control pages therefore remain privileged, but you can place them in software such as a host kernel or another privileged control component outside the hypervisor.

7.4 Hypervisor and arbiter boundary

The hypervisor and the arbiter have distinct, non-overlapping roles in the virtualization architecture. The boundary between them must remain clearly defined:

- The hypervisor owns isolation, assignment, and security policy.
- The arbiter owns scheduling policy and handover control.
- The arbiter can run outside the hypervisor.

This separation is an architectural advantage of the design. It enables scheduling policy to be implemented where it best fits the platform software stack and preserves a well-defined owner for stage-2 mappings, interrupt ownership, and privileged access control.

7.5 Guest-visible device model

From the perspective of a guest, the device model is intentionally simple.

The hypervisor typically gives a guest:

- One assigned access-window register block.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

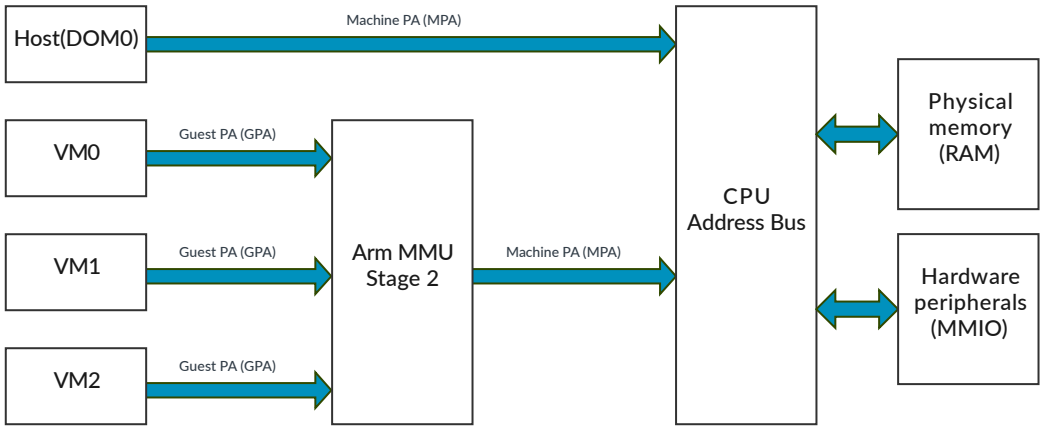
- One guest-facing interrupt for that access window.

7.6 Memory translation and isolation

The GPU provides guest-managed stage-1 translation. If the CPU side of the system uses stage-2 translation for guests, the GPU requires an external system-level translation component to provide an equivalent guest isolation boundary for GPU memory access.

The following figure shows how memory translation and isolation use the hypervisor.

Figure 7-1: Memory mapping



You must set up the ownership and responsibility model for translation layers so that:

- The guest driver owns GPU stage-1 mappings.
- The hypervisor or equivalent privileged software owns CPU stage-2 isolation.
- An external system MMU provides the GPU-side isolation needed to match that stage-2 ownership model.

In other words, the System Memory Management Unit (SMMU) provides system-level isolation. It does not replace the stage-1 GPU memory management model that the driver owns.

A design that assumes the GPU driver can support a three-stage translation model, or that tries to replace driver-owned stage-1 mappings with system MMU stage-1 behavior, does not match the software model that this guide describes.

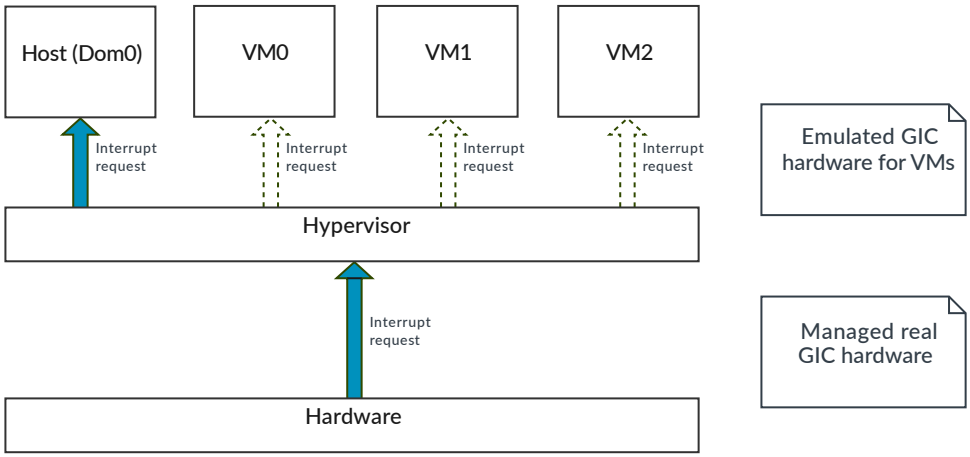
7.7 Interrupt routing

The interrupt model is simple, but it requires careful implementation.

The following figure shows how interrupt ownership maps to access window and guest assignment.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 7-2: IRQ management



Each access window has one guest-facing interrupt line. The guest assigned to that access window receives only the interrupts that belong to its guest-visible interface. Privileged pages and their associated interrupts must be routed to the same privileged domain that owns those pages.

This is less about unusual interrupt behavior and more about consistent ownership. Register ownership, interrupt ownership, and memory isolation policy must all describe the same system partitioning.

7.8 Power-management interaction

Hypervisor integration must preserve the Power-Management (PM) ownership and ordering defined for the platform. If integration changes these assumptions, runtime PM behavior becomes inconsistent across the stack.

This is especially important when GPU components are distributed across multiple VMs. Register access, interrupt routing, and runtime power-management coordination must follow one coherent ownership model.

7.9 Access-window reassignment

Some systems use access windows in a mostly static way. They assign each window when a guest is created and keep that assignment for the lifetime of the guest.

Where reassignment is supported, you must describe it as a controlled lifecycle operation:

1. Unmap the access window from the previous address space of the guest.
2. Perform the privileged access-window reset.
3. Map the access window and associated interrupts into the new owning address space of the guest.
4. Re-establish the guest-visible device model.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5. Resume normal use.

This access-window reassignment process is particularly relevant for systems that use dynamic device assignment, but it can also apply to other use cases.

7.10 Common integration mistakes

Common integration mistakes usually occur around power management, memory isolation, system MMU configuration, and cache coherency. Finding these issues early helps keep hypervisor integration predictable.

Most integration problems are not in the basic access-window concept itself, but in surrounding system integration:

- System-wide power management is not considered early enough, so the ownership model breaks when runtime power management is introduced.
- CPU stage-2 translation and GPU memory isolation are not treated as separate responsibilities.
- The system MMU is configured with the wrong translation model for the GPU-managed stage-1 memory model used by the driver.
- Cache-coherency assumptions are not handled correctly in wider system integration.

These issues are much harder to debug after the full system is assembled. For that reason, we recommend validating basic arbitration and access-window behavior in a simpler single-OS environment before moving to full hypervisor integration.

The pKVM appendices show how to achieve these responsibilities in concrete systems. For more information, see PKVM Worked Example.

7.11 Protected-memory mapping

Protected-memory handling requires the same clear ownership model as register mapping and interrupt routing.

At system level, protected-memory integration has distinct requirements. These requirements typically include the following:

- The guest driver must be able to access protected memory that it requires to boot and run the firmware correctly.
- Runtime protected memory for selected guest workloads or test applications.

From the perspective of the guest driver, both regular and protected memory regions must be available before use. The hypervisor maps these memory regions into the guest address space, while another higher-privileged software component in the system manages the associated memory permissions.

During integration, you can choose different policies to expose protected-memory to the address space of the guest.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

For example, all or part of a static protected-memory region can be exposed to the guest address space as a carveout.

If the guest cannot see the region, the driver cannot use it to run protected-mode GPU jobs. If the region is exposed incorrectly, the system risks breaking security policy, disrupting driver initialization, or placing hardware in an undefined state.

In practice, platform integration must provide the following:

- Protected memory at system level.
- A controlled mapping or access to the protected memory into the intended guest.
- A guest-visible description of the protected-memory heap so the guest software knows that it exists, and what it is for. In Linux, this is typically done using the DMA-heap framework.
- Matching system-level memory-isolation rules so the GPU access window associated with that guest can reach only the intended protected-memory heap.

The exact mechanism for mapping and assigning protected memory to a guest is hypervisor-specific. Some systems use guest configuration and reserved-memory description, while others rely more heavily on privileged firmware or VM-assignment logic. The architectural requirement is clear. The integrator must explicitly assign protected memory and must not assume it appears automatically because the guest has an access window.

This explicit protected-memory assignment requirement is especially important in CSF-based systems, where firmware entry into protected mode depends on a firmware protected-memory section being present. Guests trying to enter protected mode without a valid protected-memory assignment will not be able to run protected-mode workloads, which might result in undefined behavior.


Note

We do not prescribe a carveout size and layout, or dynamic protected memory management. However, we recomemnd that you treat these features as first-class guest-assignment resources.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8. Performance and system tuning

Performance in a virtualized GPU system depends on architectural constraints and policy decisions.

The following can influence system behavior:

- Scheduling policy
- Timeout values
- Power-management configuration

However, these mechanisms cannot guarantee behavior that the underlying architecture does not support.

The virtualization model has the greatest influence on system behavior. Partitioning provides hardware-level execution isolation, while time slicing allows multiple guests to share GPU resources over time. The appropriate model depends on the system requirements. Use partitioning when execution isolation is required. Use time slicing when resource sharing and deployment flexibility are the primary goals.

Effective performance tuning depends on representative workload profiling, realistic performance expectations, and a clear understanding of which limitations are architectural and which are policy-driven. During performance analysis, identify the main sources of latency and overhead, including the following:

- Ownership transfer
- Workload contention
- Power management
- Memory translation
- Platform-level behavior

To tune performance effectively, understand how virtualization models affect system behavior, where latency and overhead are introduced, and which tuning mechanisms influence the resulting performance characteristics.

8.1 Performance tuning considerations

Performance tuning starts by selecting the virtualization model that matches the system requirements.

If the primary goal is hard performance isolation, use hardware partitioning. If the goal is to share GPU resources across multiple guests, account for ownership transfer overhead, workload interaction, and contention for shared execution resources.

This distinction matters because many apparent performance issues result from the selected virtualization model. A time-sliced system can often be tuned to provide good practical

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

performance, but it cannot deliver the same level of isolation and predictability as a partitioned system.

8.2 Main sources of overhead

Performance analysis in a virtualized GPU system is most effective when you consider overheads separately, instead of as a single end-to-end cost.

The main sources of overhead are:

Yield latency
: Time required for the current owner to stop using the GPU and complete ownership transfer.

Reset and reopen overhead
: Time required to reset the GPU subinstance, reopen the access window, and resume execution for the next owner.

Guest contention
: Delays caused by multiple guests sharing the same execution resource.

Platform overheads
: Costs associated with interrupt handling, CPU scheduling latency, messaging paths, and memory translation.

System-level effects
: Costs associated with power management, cache coherency, and memory-bandwidth contention.

Each overhead source has different causes, owners, and mitigation options. Analyzing each source separately helps you identify the primary performance bottleneck and the most effective tuning actions.

8.3 Yield latency and performance

In a time-sliced system, the time that the current guest needs to yield the GPU is often the primary performance constraint, especially when ownership changes occur frequently.

The GPU cannot preempt shaders arbitrarily. As a result, some workloads take a long time to soft-stop before ownership can transfer to another guest. If yield times become too long, switching overhead can prevent the system from meeting latency or frame-rate targets.

Workload behavior often has a greater influence on yield time than arbitration policy. Some workloads are naturally well-suited to cooperative sharing, while others are harder to stop promptly.

For this reason, include profiling with representative workloads in your performance-tuning process. The most significant part of the ownership transfer path is often how quickly the outgoing workload yields, rather than the arbitration mechanism itself.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.4 Ownership transfer sequence

A typical ownership transfer in a time-sliced system has the following stages:

1. A waiting guest requests GPU ownership.
2. The arbiter instructs the current guest to stop.
3. The current guest soft-stops work and signals completion.
4. The platform resets and reassigns the GPU.
5. The next guest resumes execution.

Tuning considerations

The largest contributor to ownership transfer latency is typically the soft-stop or yield phase in the outgoing guest.

This behavior depends strongly on workload characteristics, so arbitration policy alone cannot fully control switching cost. Profile the system by using representative applications and workload mixes.

8.5 Sources of switching overhead

Access window context switch introduces overhead at multiple points in the software stack. Understanding these overheads helps you identify the primary contributors to switching latency and the most effective tuning opportunities.

Arbitration and message latency
: Time spent coordinating between software components during the access window switches.

Yield handling
: Time required for the current owner to stop using the GPU and complete the yield sequence.

GPU reset
: Time required to reset the GPU subinstance before access window context switch.

Access-window reopen
: Time required to reopen the access window and grant access to the next owner.

Power-state transitions
: Latency introduced by power-management operations during access window context switch.

Execution-environment resume
: Time required for the next owner to resume or restart its execution environment.

Total switching overhead depends on all these factors. Scheduling policy can influence overall system behavior, but workload characteristics, yield latency, reset costs, power-management operations, and other platform-level overheads can also contribute significantly.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.6 Performance characteristics of shared systems

In a shared, time-sliced system, guests share the same execution resource over time. As a result, each guest can affect the performance of other guests.

Practical consequences include:

- A long-running workload in one guest can increase latency for other guests.
- Fair resource sharing does not guarantee predictable performance.
- Strict frame rate or latency guarantees are difficult to achieve in a shared execution domain.
- Yield and ownership transfer overheads limit the number of guests that can share a GPU effectively.

As a result, every shared system has a practical limit on the number of guests that can meet responsiveness targets simultaneously.

Performance tuning can improve system behavior, but it cannot eliminate the effects of resource sharing. The goal is to achieve an appropriate balance between responsiveness, utilization, and expected workload mix.

8.7 Platform effects on switching latency

CPU-side and platform-side behavior can contribute significantly to owner-transfership switching latency.

Examples include:

- Guest or host drivers holding spinlocks for extended periods.
- Drivers spending excessive time in interrupt context.
- Kernel threads running for long periods without yielding.
- Limited CPU core availability in host or guest environments.
- Slow messaging paths between the arbiter and guest driver.

Time-sliced sharing depends on timely coordination between the arbiter and the guest-side driver. Delays on the CPU side can therefore increase ownership transfer latency. This can happen even when GPU-side behavior is operating as expected.

8.8 Translation and system overheads

Memory translation and isolation mechanisms contribute to overall virtualization overhead.

The GPU uses guest-managed stage-1 translation together with an external system-level isolation mechanism. As a result, performance depends on guest-visible GPU mappings. System MMU configuration, guest isolation mechanisms, and related control flows can also affect performance.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

System-level overheads can include:

- Reconfiguration costs associated with the system MMU or MPU
- Guest register mapping and access control overhead in the wider platform
- Cache coherency behavior
- Memory bandwidth contention between guests and other system components

These overheads do not change the virtualization model itself, but they can affect the performance headroom available to the system. They can also influence the effectiveness of other tuning activities.

8.9 Power management overheads

Power management can affect switching latency and overall system responsiveness.

Potential sources of overhead include:

- Powering down and powering up the GPU and related blocks
- Powering down and powering up L2 and shader cores
- Governor-driven frequency or core-state changes
- Coordination latency between software components involved in runtime power management

Performance tuning and power-management tuning should be considered together. A power-efficient configuration can still perform poorly if power-state transitions add too much latency.

8.10 Performance tuning in partitioned systems

Partitioning changes the performance characteristics of the system. It also changes the focus of performance tuning.

When workloads run on separate hardware-isolated partitions, GPU side interference between them is eliminated. However, system level contention can still occur through shared resources such as CPU time, memory bandwidth, cache behavior, and platform power management infrastructure.

As a result, performance tuning in partitioned systems focuses less on arbitration and ownership transfer behavior and more on workload placement, resource allocation, and system level contention.

Partitioning is the preferred approach when one guest's GPU performance must be isolated from another guest's workload. Time slicing is more appropriate when utilization and flexibility are more important than hard execution isolation.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.11 Performance tuning controls

You must tune multiple parts of the software stack together. Do not focus on a single parameter in isolation.

Common tuning controls

- Time-slice duration
- Arbitration policy
- Access-window and partition assignment
- DVFS and dynamic core-scaling policy
- Partition size
- How workloads are assigned to partitions

How these controls interact

Each tuning control affects the others. A short time slice combined with an unrealistic yield policy increases switching overhead. An aggressive scheduling policy can create more ownership transfer activity than your platform can tolerate.

In the Panthor reference arbiter, the `request_timeout` and `yield_timeout` parameters influence different parts of the ownership-transfer path. Your choices affect responsiveness, switching overhead, and tolerance of long-running workloads. For more information, see Arbitration timeout parameters.

Profile your workload

Profile your workload before tuning. Understand the expected mix of short-running, long-running, interactive, and background tasks. Base your tuning decisions on representative workload behavior rather than fixed parameter values.

8.11.1 Arbitration timeout parameters

The Panthor reference arbiter provides the following timeout controls:

`request_timeout`
: Controls how long the current VM can run before the round-robin scheduler attempts to switch ownership to another requester.

A shorter value can improve responsiveness for waiting guests but increases switching frequency and associated overhead.

A longer value reduces switching pressure but can increase latency for other guests.

`yield_timeout`
: Controls how long a VM can take to stop after a stop request before the system invokes `GPU_LOST` handling.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A shorter value can cause workloads that would have yielded successfully to be treated as lost too aggressively.

A longer value can allow poorly behaved or long-running workloads to delay ownership transfer for too long.

Tuning guidance

When selecting timeout values:

- Choose `request_timeout` based on the desired balance between responsiveness and switching overhead.
- Choose `yield_timeout` based on realistic stop behavior for the expected workloads.
- Validate both parameters together because the acceptable range for one depends on the value of the other.

8.12 Content profiling and workload placement

Profiling helps determine whether the selected configuration is appropriate for the expected workload.

Treat workload characteristics as a primary input to performance tuning.

Useful questions include:

- How long do typical workloads take to yield?
- How often is ownership expected to change?
- Which guests are latency-sensitive and which are throughput-oriented?
- Which workloads should be assigned to dedicated partitions rather than shared execution domains?
- Does the workload submit bounded units of work, or does it build long drain times once queued?
- Are some content types suitable for dedicated partitions but unsuitable for frequent time-sliced handover?
- What is the typical time for a workload to yield?

Workload behavior often has a greater impact on system responsiveness than adjustments to scheduling policy or timeout values.

8.13 Example tuning patterns

Base performance tuning decisions on workload characteristics and product goals rather than on fixed timeout values or scheduling policies.

Different systems place different demands on the virtualization stack. Some prioritize responsiveness for a latency-sensitive workload, while others prioritize fairness, throughput, or

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

predictable behavior across a broader range of applications. Tuning values that work well in one deployment can perform poorly in another deployment.

The most effective tuning strategies begin with an understanding of workload behavior. The following factors all influence the choice of arbitration policy and timeout values:

- Yield time
- Ownership transfer frequency
- Workload predictability
- The balance between foreground and background activity

The following examples illustrate common tuning scenarios. They show how workload characteristics affect tuning decisions and why profiling representative content is often more valuable than adjusting individual parameters in isolation.

8.13.1 Latency-sensitive guest plus background guest

Consider a system in which one guest runs a latency-sensitive workload, for example a user-facing display application, while a second guest runs less time-critical background work.

In this scenario, the tuning goal is to reduce visible latency for the foreground guest without creating excessive switching overhead. A shorter `request_timeout` can help the waiting foreground guest regain GPU ownership more quickly, but only if the background workload can also yield promptly.

If the background workload is slow to soft-stop, then reducing `request_timeout` alone can increase the frequency of ownership transfer attempts without improving end-user responsiveness.

A practical approach is to profile the yield behavior of the background workload first. Then select a `yield_timeout` that reflects realistic stop times. Reduce `request_timeout` only to the point where responsiveness improves without unnecessary switching activity.

8.13.2 Controlled and open workload environments

Consider a system with a tightly controlled workload mix, where you know the expected applications in advance. Compare it with a system that must support a wider range of user-supplied or unpredictable workloads.

Your tuning goal is to select timeout values and scheduling policies that provide good responsiveness without creating unnecessary recovery events or ownership transfer overhead. A controlled workload mix makes this easier because you can profile representative content and establish realistic stop-time distributions.

If the workload mix is less predictable, timeout values typically need to be more conservative. A wider range of workload behavior increases the likelihood of encountering long-running or slow-to-yield content that was not considered during initial tuning.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Use a practical approach: profile the expected workload mix first, then tune the system based on realistic workload behavior. When you cannot tightly control the workload mix, choose timeout values that tolerate a broader range of content and validate them by using representative worst-case scenarios.

8.14 Trace-based profiling

Base performance tuning on workload characteristics and product goals rather than on fixed timeout values or scheduling policies.

Use a profiling view that combines multiple sources of information, including:

- Protocol messages
- Access-window events
- Ownership transfer activity
- Guest-driver behavior
- Recovery events

Correlate these events to determine whether workload behavior, software latency, or wider platform effects cause a performance issue.

For example, trace data can show whether slow workload yield behavior causes a long ownership transfer. Trace data can also show whether CPU-side latency in the guest or host causes the delay, or whether power-management activity introduces it.

Trace-based profiling is particularly valuable when tuning ownership transfer behavior because similar symptoms can have very different underlying causes.

8.14.1 Trace sources for virtualization profiling

The following trace sources are useful when profiling a virtualized GPU system:

Arbiter-to-guest protocol messages
: Messages exchanged between the guest-side driver and the arbitration layer.

Access-window events
: Access-window state transitions and interrupts, including `WINDOW_OPENED`.

Partition-control activity
: Commands such as `YIELD_NOW`, `YIELD_IDLE`, `OPEN_WINDOW`, and `CLOSE_WINDOW`.

Guest-driver activity
: Guest-side driver suspend and resume operations.

Recovery events
: Timeout and recovery behavior, including `GPU_LOST`.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Panthor tracepoint families

The current Panthor stack provides the following tracepoint families:

Access-window tracepoints

Access-window message and state tracepoints in the main `panthor` driver.

Arbitration tracepoints

Arbitration message, state, and partition-command tracepoints in `panthor_arbitration`.

Power-management tracepoints

Runtime power-management, dynamic voltage and frequency scaling (DVFS), and core-mask tracepoints.

Usage considerations

These trace sources are particularly useful when tuning `request_timeout` and `yield_timeout`. Similar user-visible behavior can result from different causes within the ownership-transfer path.

8.14.2 Analyzing system behavior with trace data

Trace data can provide visibility into behavior at the arbiter, guest-driver, and system levels.

Arbiter level

Trace data can show:

- When a guest requested GPU ownership
- When a stop request was issued
- Whether the outgoing guest responded promptly
- When the partition-control path issued `YIELD_NOW`, `YIELD_IDLE`, `OPEN_WINDOW`, or `CLOSE_WINDOW`
- When the next guest was granted ownership

Guest-driver level

Trace data can show:

- Whether the access window moved through the expected ownership states
- Whether the guest received the expected protocol messages
- Whether suspend and resume operations completed successfully
- Whether ownership transfer completed normally or required timeout-driven recovery

System level

Trace data can show:

- Whether runtime power management (PM) or system PM transitions are increasing ownership transfer latency
- Whether governor activity is changing frequency or core availability during ownership transfer
- Whether the platform is spending more time in reset and reopen operations than in yield handling

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.14.3 Measuring ownership transfer latency

Analyzing ownership transfer latency requires more than measuring total switch time. Break the ownership transfer sequence into individual stages to identify the primary sources of latency and the most effective tuning opportunities.

A typical ownership transfer analysis considers the following events:

1. A guest requests GPU ownership.
2. The arbiter issues a stop or yield-related command to the current owner.
3. The current owner responds, or the access window changes state.
4. The partition is reset or reopened.
5. The next guest reaches `GPU_GRANTED` or the access window is reopened.

This approach enables you to divide the total ownership transfer time into:

- Message and software coordination latency
- Workload-dependent yield latency
- Reset and reopen latency
- Power-management or governor overhead

This breakdown is often more useful than a single end-to-end ownership transfer measurement because it identifies the part of the stack that contributes most to the observed latency.

8.14.4 Profile representative content

Trace-based profiling is most effective when you use representative workloads rather than simple demonstrations or synthetic tests.

Useful comparisons include:

- Short, well-bounded workloads versus long-running workloads
- Latency-sensitive guests versus throughput-oriented guests
- Low-contention scenarios versus realistic mixed-load scenarios
- Normal ownership transfer versus cases that approach `yield_timeout`

Use these comparisons to identify whether performance limitations are caused by workload behavior, arbitration policy, or wider platform integration effects.

Trace-based profiling with representative workloads helps you determine:

- Whether the system is slow
- Why the system is slow
- Which part of the system is responsible for the observed behavior

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.15 Architectural performance constraints

Architecture determines some limits, regardless of configuration or tuning choices.

Examples include:

- A shared execution resource cannot provide hard performance isolation between guests.
- Long-running workloads can dominate yield latency.
- Ownership transfer overhead cannot be reduced to zero.
- Increasing the number of guests eventually reduces achievable responsiveness.
- Poor platform integration can negate the benefits of an otherwise well-tuned GPU software stack.

Understanding these limits is an important part of performance tuning. Effective tuning starts with distinguishing architectural constraints from policy choices and configuration decisions.

Incorrect expectations often cause more integration issues than incorrect timeout or scheduling values.

8.16 Tune virtualization performance

Tune virtualization performance for the target workload and deployment model.

Procedure

1. Choose the virtualization model that best matches the product requirements.
2. Measure ownership transfer behavior using representative workloads.
3. Separate yield latency, reset cost, platform latency, and power-management effects.
4. Tune scheduler policy, time-slice duration, and timeout values together.
5. Re-test using the expected workload mix rather than simple demonstrations.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

9. Reference software stack

This chapter describes the Linux reference software stack for GPU virtualization on Panthor. It explains how the main software modules fit together in a virtualized system. It identifies which module is responsible for each hardware function and how the modules interact during system bring-up.

Use this chapter to understand:

- Which module owns the guest-visible access window
- Which modules operate with privileged access
- How the system, governor, arbitration, and guest-facing drivers interact
- The dependency and bring-up order used by the Linux reference implementation

For detailed information about the Panthor driver see the *Arm® Mali™ Immortalis™ GPU v10 Architecture and Later DDK Integration Manual*. This document includes information about:

- Build procedures
- Kernel configuration
- Device tree examples
- Interface definitions
- Module parameters
- Tracing
- Testing

9.1 Source layout

In the Linux reference implementation, the virtualization stack is split across the following Panthor source directories:

`drivers/gpu/drm/panthor/`
: Guest-visible Panthor driver, including access-window handling, firmware integration, scheduling, memory management, and generic Command Stream Frontend (CSF) functionality.

`drivers/gpu/drm/panthor/system/`
: Privileged Access Manager (AM) system support, including the system driver and optional assignment-block support.

`drivers/gpu/drm/panthor/arbitration/`
: Arbitration driver, resource-group handling, partition-control handling, arbiter core functionality, scheduling logic, and arbiter sysfs support.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

drivers/gpu/drm/panthor/governor/
: Standalone governor driver for devfreq, IPA-based power estimation, and dynamic core-mask management.

Build options

CONFIG_DRM_PANTHOR
: Builds the main Panthor driver.

CONFIG_DRM_PANTHOR_VIRT
: Builds the virtualization-specific `system/` and `arbitration/` modules.

CONFIG_DRM_PANTHOR_GOVERNOR
: Builds the standalone governor module.

CONFIG_DRM_PANTHOR_FW_SCHED
: Selects the firmware-based scheduling path within the main Panthor driver. This option is mandatory on virtualization-ready Arm Mali™ CSF GPUs.

9.2 Main modules

The Linux reference virtualization stack is split into 4 main modules. These modules provide the following functions:

- Guest-visible GPU access
- Privileged system control
- Arbitration
- Power-management functionality

The modules operate together as a layered stack. The privileged modules establish system control, power management, and arbitration services before guest-visible GPU drivers can operate correctly.

The Linux reference stack uses the following ownership model:

panthor_system.ko
: Manages privileged Access Manager system configuration.

panthor_governor.ko
: provides DVFS and dynamic core-scaling functionality.

panthor_arbitration.ko
: Manages GPU ownership, access-window assignment, and arbitration policy. It also owns scheduling, timeout handling, and ownership transfer.

panthor.ko
: Provides the guest-visible GPU driver associated with an access window.

This separation allows guest-visible GPU operation, arbitration policy, power management, and system-level control to evolve independently while preserving clear ownership boundaries.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

9.2.1 panthor.ko

The `panthor.ko` module enables GPU workload submission and implements the guest-side virtualization interface. The `panthor.ko` module also manages GPU request handling, yield operations, and protected memory support.

Source location

`drivers/gpu/drm/panthor/`

Purpose

Provides the guest-visible GPU driver bound to an access window.

The driver:

- Submits work through the access-window-backed register interface when the access window is open.
- Implements the guest-side arbiter-VM messaging path.
- Tracks access-window assignment state, handshake state, GPU request flow, and guest-side stop handling.
- Integrates CSF firmware boot, queue management, runtime power management, and protected-memory support.

Implementation notes

Access-window-specific logic

Implemented in `panthor_aw.c`.

Protocol messaging

Uses definitions from `include/am/panthor_arb_vm_protocol.h`.

Supported protocol messages

Sends `VM_ARB_GPU_REQUEST` and `VM_ARB_GPU_STOPPED`, handles `ARB_VM_INIT` and `ARB_VM_GPU_STOP`, and responds to access-window interrupts.

Protected-memory integration

Controlled through the `protm_heap_name` module parameter in `panthor_device.c`.

Usage considerations

Driver instance model

One `panthor.ko` instance is normally associated with one guest-visible access window.

Probe ordering

The driver probes only after all required supplier devices are available and the access window is enabled for the target partition.

Assignment requirements

If the access window is not assigned to a valid arbitration path, the driver can defer probing or time out while waiting for GPU access.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Protected memory

Protected-memory support is provided through a protected Linux DMA heap, providing an abstraction to the vendor specific trusted zone implementation.

9.2.2 panthor_system.ko

The `panthor_system.ko` module provides preferred Access Manager system support. It manages system-level GPU configuration, privileged interrupt handling, and assignment-related configuration where supported.

Source location

`drivers/gpu/drm/panthor/system/`

Purpose

Provides the privileged system-control path for the virtualization stack.

The module:

- Owns the privileged Access Manager (AM) system register interface.
- Manages system-facing GPU and Access Manager configuration.
- Handles privileged interrupt masking and status processing for the AM system block.
- Saves and restores privileged configuration across suspend and resume.
- Optionally supports assignment-block functionality on products that expose the assign block.

Implementation notes

Core driver

Implemented in `panthor_system_drv.c`.

Assignment support

Implemented in `panthor_assign.c`.

Initialization

Initializes clocks and optional regulators, applies configuration from device tree, and integrates with runtime power management.

Usage considerations

Probe ordering

The module probes before the arbitration and guest-facing drivers.

Responsibilities

This module is responsible for privileged Access Manager system configuration rather than guest-visible job submission.

Assignment configuration

When the assignment block is present, the module parses assignment-related device tree properties and applies them to the hardware.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Device tree information

Detailed system and assignment configuration properties are documented in the Panthor integration manual.

9.2.3 panthor_arbitration.ko

The `panthor_arbitration.ko` module provides the privileged arbitration path for GPU virtualization. It manages access-window assignment, ownership transfer, GPU-subinstance scheduling, and the interaction between guest-side requests and privileged resource management.

Source location

`drivers/gpu/drm/panthor/arbitration/`

Purpose

Provides the privileged arbitration and ownership-management path.

The module:

- Owns the privileged arbitration path.
- Binds the partition-control and resource-group hardware blocks.
- Implements the arbiter-side VM model, access-window assignment tracking, and GPU-subinstance scheduling.
- Provides the reference round-robin scheduling policy.
- Exposes arbiter configuration and partition-assignment controls through module parameters and sysfs.

Implementation notes

Driver entry points

Implemented in `panthor_arbitration_drv.c` and `panthor_arbitration_core.c`.

Arbiter core

Implemented in `panthor_arbiter_core.c`.

GPU-subinstance handling

Implemented in `panthor_arbiter_gpu_subinstance.c`.

Reference scheduler

Implemented in `panthor_arbiter_scheduler_rr.c`.

sysfs support

Implemented in `panthor_arbiter_sysfs.c`.

Usage considerations

Probe ordering

This module is available before guest-facing `panthor.ko` instances attempt to request GPU ownership.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Scheduling ownership

This module owns the privileged scheduling policy in the Linux reference implementation.

Integration role

This module acts as the integration point between the guest-side messaging path and the privileged VM-assignment path.

9.2.4 panthor_governor.ko

The `panthor_governor.ko` module provides governor functionality for GPU power and performance management. It integrates devfreq, IPA-based power estimation, and dynamic core scaling into the virtualization software stack.

Source location

`drivers/gpu/drm/panthor/governor/`

Purpose

Provides the governor path for GPU frequency and power management.

The module:

- Implements standalone devfreq-based DVFS and dynamic core scaling.
- Integrates IPA counter collection and power-model support.
- Supports both active and passive governor configurations which allows Operating Power Points (OPPs) to be defined for each clock and supply combination across power domains and sub-power domains.
  - Active controls the governor and parent clock and power supply.
  - Passive controls the children clock and power supply, under the parent.
- Provides Linux support for governor-side dependency parsing through Device Tree `deps` supplier links.

Implementation notes

Platform driver

Implemented in `panthor_governor_drv.c`.

Device and dependency handling

Implemented in `panthor_governor_device.c`.

Core-mask handling

Implemented in `panthor_governor_core_mask.c`.

IPA integration

Implemented in the `panthor_governor_ipa*.c` source files.

Usage considerations

Active mode

The governor monitors utilization and adjusts GPU frequency dynamically.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Passive mode

The governor follows a parent devfreq instance. On utilisation increase, the parent updates the OPP first, then the child. On utilisation reduction, the child updates the OPP first, then updates the parent.

Dependency management

The driver parses `deps` supplier relationships and creates Linux device links so that probe ordering and runtime power-management ordering follow the intended hierarchy.

Additional information

Detailed governor bindings, devfreq interfaces, and IPA-specific controls are documented in the Panthor integration manual.

9.3 Integration model

The Linux reference stack uses separate Device Tree nodes for the major software components in the virtualization stack.

A typical deployment includes:

- One `panthor-system` node
- One `panthor-arbitration` node for each arbitration instance
- One or more `panthor-governor` nodes, depending on the governor topology
- One `gpu@...` node for each guest-visible access window

The reference implementation also uses dependency descriptions to derive probe ordering and runtime power-management ordering from the ownership hierarchy.

The exact Device Tree syntax is documented in the Panthor integration manual. The important architectural requirement is that the dependency graph is described explicitly enough to ensure that:

- Probe ordering is correct
- Deferred probe behavior is predictable
- Runtime power-management ordering follows the ownership hierarchy

9.4 Probe and module load order

The Linux stack expects the following order:

1. `panthor_system.ko`
2. `panthor_governor.ko`
3. `panthor_arbitration.ko`
4. `panthor.ko`

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Reference module load sequence

If the stack is built as loadable modules, use the following load order:

```
modprobe panthor_system
modprobe panthor_governor
modprobe panthor_arbitration request_timeout=1000 yield_timeout=1000
modprobe panthor
```

Built-in configurations

If the stack is built into the kernel, Linux determines probe timing through platform-device creation and deferred probe.

Although this configuration does not require manual module loading, the dependency model, deferred probe behavior, and runtime power-management ordering must still follow the same ownership hierarchy.

9.4.1 Deferred probe behavior

Deferred probe can occur when required dependencies are not yet available.

Common causes include:

- A supplier device named in `deps` has not probed yet.
- The guest-facing access window is visible, but the arbitration path is not ready.
- The access window is not enabled for the partition that should own it.

An access window that is not enabled through `assigned_access_windows` can fail to probe with `-ETIMEDOUT`.

In a virtualized deployment, probe failures are often related to ownership or dependency configuration, not local driver faults. When you investigate a probe failure, verify ownership assignments and dependency relationships before treating the issue as a guest-driver problem.

9.5 Linux reference stack overview

The Linux reference stack separates guest-visible GPU functionality from privileged control functions.

The stack consists of:

- `panthor_system.ko` for privileged system-level GPU and Access Manager configuration
- `panthor_arbitration.ko` for ownership assignment, arbitration, and scheduling
- `panthor_governor.ko` for power and performance management, where used
- `panthor.ko` instances for guest-visible access-window interfaces

The stack relies on explicit ownership assignment, dependency ordering, and runtime power-management relationships between modules.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The virtualization-specific aspects of the stack include:

- Access-window enablement
- Ownership assignment
- Arbitration
- Protected-memory integration

For detailed Panthor driver information, including module parameters, sysfs interfaces, tracing, and device-tree bindings, refer to the companion Panthor integration manual.

9.5.1 Initialize the Linux reference stack

Initialize the Linux reference stack in layers so that the privileged infrastructure is available before enabling guest-visible GPU access.

Before you begin

Before you can initialize the linux reference stack, you must turn on the clocks and regulators.

Procedure

1. Initialize the system `panthor_system.ko` and arbitration infrastructure `panthor_arbitration.ko`.
2. If you are using `panthor_governor.ko`, enable it.
3. Assign access windows to the intended partitions.
4. Initialize the guest-facing `panthor.ko` instances.

9.5.2 Validate Linux stack initialization

Validate that the Linux reference stack has initialized correctly.

Procedure

1. Confirm that the system and arbitration nodes probe successfully.
2. Confirm that the governor node probes successfully, if the governor is enabled.
3. Confirm that the intended access windows are enabled for the intended partition.
4. Confirm that the guest-facing `panthor.ko` instances probe successfully.
5. Confirm that the Arbiter-VM handshake completes successfully.
6. Confirm that `VM_ARB_GPU_REQUEST` messages can be serviced.

Results

The Linux reference stack initializes successfully, guest-visible drivers can communicate with the arbitration layer, and GPU ownership requests can be processed correctly.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Appendix A Single OS worked example

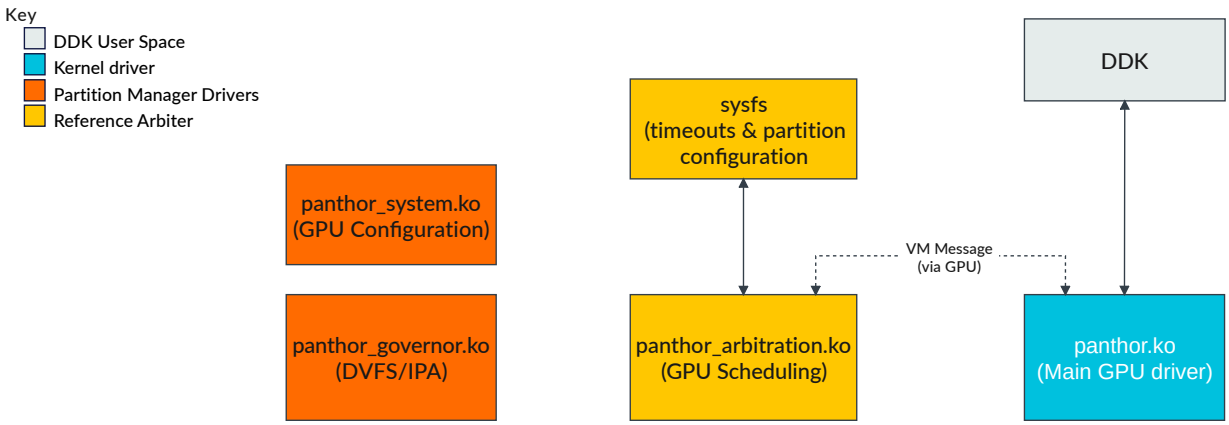
This appendix provides a worked example of a single-OS deployment with multiple access windows.

The example is intentionally simple. It is not intended to represent a production virtualization design. Instead, it provides a straightforward environment for initial bring-up and validation.

This configuration is often a useful starting point because it enables you to validate the access-window protocol and arbitration flow. The configuration also helps you validate owner-transfer behavior, key guest-driver, and firmware interactions before you introduce additional virtualization components.

These additional components can include hypervisor integration, stage-2 address translation, and VM lifecycle management.

Figure A-1: Single CPU cluster configuration with single OS



A.1 Scope of the single-OS example

This appendix describes a deployment where:

- A single operating system owns the complete GPU software stack.
- Multiple access windows are visible within the same operating system.
- You can use 1 access window for local validation while you exercise another through the arbitration path.
- The privileged system, governor, arbitration, and guest-facing driver functions run in the same software environment.

This example does not imply that production systems must use a single-OS deployment model. We include this model because it is the simplest way to validate the architecture before you integrate it into a wider system.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A.2 Why this environment is useful

The single-OS environment removes several sources of integration complexity.

These sources are:

- Hypervisor-specific device assignment
- CPU stage-2 address translation configuration
- Complex interrupt ownership relationships
- Additional runtime interactions between virtualization layers

As a result, you can observe runtime behavior with fewer moving parts and less platform-specific interference.

This makes the environment suited to answering common bring-up questions, for example:

- Does the access-window handshake complete successfully?
- Does GPU request and grant behavior work as expected?
- Do `WINDOW_OPENED` and `WINDOW_CLOSED` occur at the expected points in the ownership-transfer flow?
- Do normal yield, idle-yield, and timeout-recovery paths behave correctly?
- Does queued work persist across time slices as expected?

A.3 Single-OS system deployment

In this worked example, a single operating system hosts:

- The privileged system-control path
- The governor function
- The arbitration-side control path
- 2 or more access windows
- 1 local guest-facing driver instance for each access window under test

The simplest arrangement typically uses 1 access window for basic bring-up and validation. The arrangement then introduces a second access window to exercise switching behavior, yield behavior, and queue persistence under arbitration.

This arrangement enables validation of the ownership-transfer model and arbitration behavior without introducing the additional complexity of a hypervisor or virtual-machine lifecycle management.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A.4 Initialize a single-OS deployment

Use a single-OS deployment to validate the GPU virtualization infrastructure in a simple software environment. This approach verifies the basic control path and ownership-transfer behavior before introducing additional software complexity, such as a hypervisor or advanced scheduling.

Procedure

1. Probe the privileged modules.
2. Confirm that you can access the system-level GPU blocks.
3. Initialize 1 access window.
4. Confirm that the guest-facing driver probes and boots the firmware.
5. Add a second access window.
6. Confirm that the software sees both access-window interfaces.
7. Initialize the arbitration-side logic.
8. Confirm that GPU request and grant behavior works correctly.
9. Exercise ownership transfer between the access windows, including open, close, yield, and recovery behavior.
10. Validate the basic control path.
11. Add tracing and performance instrumentation.

Results

The platform validates the hardware ownership-transfer model in a simple software environment.

A.5 Validation checklist

Use these validation checks during early initialization to confirm that the basic ownership-transfer and control paths operate correctly. Establishing this baseline before introducing virtualization or other advanced features simplifies debugging and helps isolate configuration and integration issues.

Required validation areas

Validate the following areas to confirm that access-window operation, ownership transfer, and guest-driver behavior are functioning correctly before introducing more complex system configurations.

Access-window initialization and handshake

Confirm that access-window initialization and handshake complete successfully.

GPU request and grant behavior

Confirm that ownership requests and grants occur as expected.

`WINDOW_OPENED` and `WINDOW_CLOSED` sequencing

Confirm that ownership transitions are signaled in the correct order.

Yield handling

Confirm normal yield operation and yield-timeout handling.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Queue persistence

Confirm that queued work persists correctly across time slices.

Idle-to-active transitions

Confirm correct behavior when the GPU becomes idle and is later requested again.

Asynchronous yield validation

If the platform supports the asynchronous yield path, validate the preferred ownership-transfer sequence:

1. `YIELD_NOW` or `YIELD_IDLE`
2. `YIELD_DONE`
3. `WINDOW_CLOSED`

Before you introduce the additional complexity of a multi-VM deployment, validate this sequence.

A.6 Relationship to later integration

Treat the single-OS worked example as a staging environment for later appendices, not as a competing deployment model.

This environment is useful because it enables you to establish that:

- The hardware-facing control path is wired correctly
- The guest-facing driver behavior is sound
- The arbitration flow is understandable in logs and traces
- The basic timeout and recovery settings are reasonable for the intended content

After validating the basic control path, you can extend the system to support:

- Hypervisor-based assignment
- Stage-2 memory translation
- Wider runtime power-management coordination
- Protected-memory exposure where required

A.7 Shared-partition GPU deployment

A shared-partition GPU time-slices access between multiple access windows.

In this model, multiple guest-visible access windows exist, but only 1 access window owns the shared GPU subinstance at a time.

The arbiter grants ownership, forces yield when needed, and reopens the GPU subinstance for the next access window.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A single-OS configuration is often the simplest way to initialize the platform. It enables validation of access-window initialization, arbitration behavior, ownership transfer, and firmware interaction before you introduce virtualization-specific complexity.

After 1 access window initializes, boots firmware, and submits work successfully, you can add additional access windows. This enables you to exercise arbitration and ownership-transfer behavior.

The arbitration driver controls ownership transfers. The driver also manages requests, grants, yields, and recovery operations for the shared GPU subinstance.

In a single-OS environment for a shared-partition GPU, the arbitration driver supports the following configuration options.

Module parameters:

- `request_timeout`
  - Time, in milliseconds, that the GPU is assigned to an access window.
- `yield_timeout`
  - Time, in milliseconds, within which the access window must yield the GPU before the window is forcibly closed.

SysFS entries:

- `/sys/bus/platform/devices/*.panthor-arbitration/arbiter.0/partitions/partition0/assigned_access_windows`
  - Bitmask of access windows assigned to the shared partition. This is `0xFFFF` by default, meaning that all 16 access windows are assigned to this partition.
  - Shared-partition systems can default all access windows to one partition because there is only one GPU partition to target.
  - System integrators can still reduce that mask if only a subset of access windows should be exposed.
  - The Panthor driver for an access window that is not assigned can fail to probe with `-ETIMEDOUT`.

A.7.1 Shared-partition GPU device-tree example

To probe Panthor with the virtualization software stack in a single-OS environment, use the following device-tree example.

The example includes:

- The privileged system node
- 1 arbitration node
- 1 guest-visible access window

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The example omits the governor to keep the focus on basic probe and ownership-transfer behavior rather than power-policy integration.

```
/ {
    gpu_clock: gpu-clock {
        #clock-cells = <0>;
        compatible = "fixed-clock";
        clock-frequency = <50000000>;
    };

    gpu_supply: gpu-supply {
            compatible = "regulator-fixed";
            regulator-name = "gpu-supply";
            regulator-min-microvolt = <650000>;
            regulator-max-microvolt = <650000>;
    };

    gpu_system: panthor-system@0 {
        compatible = "arm,mali-gen5-am-system";
        reg = <0x0 0x0 0x0 0x10000>;
        interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&gpu_clock>;
        clock-names = "gpu";
        gpu-supply = <&gpu_supply>;
    };

    panthor_arbitration: panthor-arbitration@50000 {
        compatible = "arm,mali-gen5-am-arbitration";
        reg = <0x0 0x50000 0x0 0x10000>,
              <0x0 0xc0000 0x0 0x10000>;
        reg-names = "pc0", "rg0";
        interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "pc0", "rg0";
        deps {
            suppliers {
                supplier@0 {
                    remote = <&gpu_system>;
                };
            };
        };
    };

    aw0: gpu@100000 {
        compatible = "arm,mali-gen5-am";
        reg = <0x0 0x100000 0x0 0x480000>;
        clocks = <&gpu_clock>;
        interrupts = <GIC_SPI 3 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "IRQAW";
        deps {
            status = "disabled";
            suppliers {
                supplier@0 {
                    remote = <&panthor_arbitration>;
                };
            };
        };
    };
};
```

Additional access windows

After basic initialization succeeds, you can add additional access windows to exercise arbitration and ownership transfer. The following example shows the code you can add to do this:

```
    aw1: gpu@580000 {
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
        compatible = "arm,mali-gen5-am";
        reg = <0x0 0x580000 0x0 0x480000>;
        clocks = <&gpu_clock>;
        interrupts = <GIC_SPI 4 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "IRQAW";
        deps {
            status = "disabled";
            suppliers {
                supplier@0 {
                    remote = <&panthor_arbitration>;
                };
            };
        };
    };
```

A.7.2 Initialize a shared-partition GPU deployment

Initialize a shared-partition GPU deployment in a single-OS environment.

Procedure

1. Load the privileged system module.
   `modprobe panthor_system`
2. Load the arbitration module.
   `modprobe panthor_arbitration request_timeout=1000 yield_timeout=1000`
3. Load the guest-facing driver.
   `modprobe panthor`

Results

After the drivers probe successfully, you can see the following messages in the `dmesg` log.

`panthor_system.ko`

```
[   60.446962] panthor_system 0.panthor-system: probed
```

`panthor_arbitration.ko`

```
[   60.821370] platform arbiter.0: P0: Created gpu-subinstance with aw_mask=0xffff
```

`panthor.ko`

```
[   61.102526] panthor 100000.gpu: [drm] core clock id 0 rate = 10000000
[   61.103081] panthor 100000.gpu: [drm] No coregroup clock in dts
[   61.103191] panthor 100000.gpu: [drm] No stacks clock in dts
[   61.103235] panthor 100000.gpu: [drm] No neural clock in dts
[   61.675452] panthor 100000.gpu: [drm] System coherency is set to NONE
[   61.676574] panthor 100000.gpu: [drm] Mali-G2-Pro id 0xf0805 major 0x0 minor 0x0
 status 0x0
[   61.676702] panthor 100000.gpu: [drm] Features: L2:0x7120306 Tiler:0x809 Mem:0x1
 MMU:0x82830 AS:0xff
[   61.676841] panthor 100000.gpu: [drm] shader_present=0x1 l2_present=0x1
 tiler_present=0x1 neural_present=0x1
[   61.702163] panthor 100000.gpu: [drm] Firmware protected mode entry supported
[   61.721329] panthor 100000.gpu: [drm] Firmware protected mode entry supported
[   61.739057] panthor 100000.gpu: [drm] Firmware protected mode entry supported
[   61.755489] panthor 100000.gpu: [drm] Firmware git sha: ....
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
[   61.890716] panthor 100000.gpu: [drm] CSF FW using interface v2.1.0.0, Features
 0x10 Instrumentation features 0x71
[   61.898403] panthor 100000.gpu: [drm] Performance counter subsystem initialized
[   61.901255] [drm] Initialized panthor 1.7.0 for 100000.gpu on minor 2
```

These messages indicate that:

- The privileged system module probed successfully.
- The arbitration driver created a shared GPU subinstance for the partition.
- The guest-facing Panthor driver booted the firmware successfully through the access window.

A.7.3 Shared-partition arbitration configuration

In a single-OS environment for a shared-partition GPU, the arbitration driver supports the following configuration options.

Module parameters

The arbitration driver provides the following module parameters to control GPU ownership duration and yield timing.

`request_timeout`
: Time, in milliseconds, for which the GPU remains assigned to an access window.

`yield_timeout`
: Time, in milliseconds, before the access window must yield the GPU, after which the window is forcibly closed.

SysFS entries

`/sys/bus/platform/devices/*.panthor-arbitration/arbiter.0/partitions/partition.0/assigned_access_windows`
: Bitmask of access windows assigned to the shared partition.

By default, this value is `0xFFFF`, which assigns all 16 access windows to the partition.

Shared-partition systems can assign all access windows to a single partition because there is only one GPU partition.

You can reduce the mask if only a subset of access windows must be exposed.

The Panthor driver for an access window that is not assigned can fail to probe with `-ETIMEDOUT`.

A.8 Multi-partition GPU deployment

A partition-capable GPU can enable multiple access windows to run concurrently.

In this model, the hardware can expose more than 1 partition-backed GPU subinstance at the same time.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Therefore, validation must cover both access-window assignment and partition-resource assignment.

The privileged software is responsible for assigning slices, partitions, resource groups, and access windows so that you can create guest-visible GPU instances.

Unlike a shared-partition deployment, ownership transfer is not the only concern.

You must also verify that partition resources are assigned correctly before guest-facing drivers probe.

A typical deployment includes a privileged system-control node, 1 or more arbitration instances, and 1 or more guest-visible access windows.

After you validate the basic initialization and ownership-transfer behavior, you can add the governor later.

A.8.1 Multi-partition GPU device-tree example

The following device-tree example probes Panthor with the virtualization software stack in a single-OS environment for a multi-partition GPU.

The example includes:

- A privileged system node that manages GPU-wide configuration and resource assignment.
- One arbitration node.
- One guest-visible access window.

The example omits the governor to keep the focus on probe and ownership-transfer behavior.

Device-tree requirements

Partition-capable GPUs require additional device-tree configuration.

`panthor-system`

Include the `AM_SYSTEM` register range at index 0 of the `reg` property and the `AM_ASSIGN` register range at index 1.

`panthor-arbitration`

Provide register ranges and interrupts for each supported partition-control block and resource group.

Example device tree

The following example shows a device tree:

```
/ {
    gpu_clock: gpu-clock {
        #clock-cells = <0>;
        compatible = "fixed-clock";
        clock-frequency = <50000000>;
    };
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
    gpu_supply: gpu-supply {
            compatible = "regulator-fixed";
            regulator-name = "gpu-supply";
            regulator-min-microvolt = <650000>;
            regulator-max-microvolt = <650000>;
    };

    gpu_system: panthor-system@0 {
        compatible = "arm,mali-gen5-am-system";
        reg = <0x0 0x0 0x0 0x10000>,
              <0x0 0x00010000 0x0 0x10000>;
        interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&gpu_clock>;
        clock-names = "gpu";
        gpu-supply = <&gpu_supply>;
    };

    panthor_arbitration: panthor-arbitration@50000 {
        compatible = "arm,mali-gen5-am-arbitration";
        reg = <0x0 0x40000 0x0 0x10000>,
              <0x0 0x50000 0x0 0x10000>,
              <0x0 0x60000 0x0 0x10000>,
              <0x0 0x70000 0x0 0x10000>,
              <0x0 0xc0000 0x0 0x10000>,
              <0x0 0xd0000 0x0 0x10000>,
              <0x0 0xe0000 0x0 0x10000>,
              <0x0 0xf0000 0x0 0x10000>;
        reg-names = "pc0", "pc1", "pc2", "pc3",
                    "rg0", "rg1", "rg2", "rg3";
        interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 3 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 4 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 5 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 6 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 7 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 8 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 9 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "pc0", "pc1", "pc2", "pc3",
                          "rg0", "rg1", "rg2", "rg3";
        deps {
            suppliers {
                supplier@0 {
                    remote = <&gpu_system>;
                };
            };
        };
    };

    aw0: gpu@100000 {
        compatible = "arm,mali-gen5-am";
        reg = <0x0 0x100000 0x0 0x480000>;
        clocks = <&gpu_clock>;
        interrupts = <GIC_SPI 10 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "IRQAW";
        deps {
            status = "disabled";
            suppliers {
                supplier@0 {
                    remote = <&panthor_arbitration>;
                };
            };
        };
    };
};
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Additional access windows

After one access window has initialized, booted firmware, and submitted work successfully, you can add additional access windows to exercise arbitration and ownership-transfer behavior.

```
    aw1: gpu@580000 {
        compatible = "arm,mali-gen5-am";
        reg = <0x0 0x580000 0x0 0x480000>;
        clocks = <&gpu_clock>;
        interrupts = <GIC_SPI 11 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "IRQAW";
        deps {
            status = "disabled";
            suppliers {
                supplier@0 {
                    remote = <&panthor_arbitration>;
                };
            };
        };
    };
```

A.8.2 Initialize a multi-partition GPU deployment

Initialize a multi-partition GPU deployment in a single-OS environment.

Procedure

1. Load the privileged system module.
   `modprobe panthor_system`
2. Load the arbitration module.
   `modprobe panthor_arbitration request_timeout=1000 yield_timeout=1000`
3. Assign slices to a partition.
   `echo 0x1 > /sys/bus/platform/devices/*.panthor-arbitration/arbiter.0/partitions/partition.0/assigned_slices`
4. Assign access windows to the same partition.
   `echo 0xFFFF > /sys/bus/platform/devices/*.panthor-arbitration/arbiter.0/partitions/partition.0/assigned_access_windows`
5. Load the guest-facing driver.
   `modprobe panthor`

Results

The guest-facing Panthor driver probes successfully against a valid GPU subinstance.

Partition resources and access-window assignments are available before firmware boot begins.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A.8.3 Example probe messages

The following messages indicate successful probe and initialization of a multi-partition GPU deployment.

panthor_system.ko

The `panthor_system.ko` module reports the initial resource-group configuration during probe. Review these messages to confirm that the expected partitions, slices, access windows, and AXI bus assignments were initialized successfully.

```
[   33.996205] panthor_system 80010000.panthor-system: Resource Group Assignment:-
[   33.996205] RG0 BUS[A] S[0 1 2 3] P[0 1 2 3] W[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
 15]
[   33.996205] RG1 BUS[A] S[       ] P[       ] W[
   ]
[   33.996205] RG2 BUS[A] S[       ] P[       ] W[
   ]
[   33.996205] RG3 BUS[A] S[       ] P[       ] W[
   ]
```

By default:

- All partitions are assigned to resource group 0.
- All slices are assigned to resource group 0.
- All access windows are assigned to resource group 0.
- All resource groups are assigned to AXI bus A.

panthor_arbitration.ko

The `panthor_arbitration.ko` module reports the GPU subinstances created during arbitration initialization. Review these messages to confirm that the expected partitions were created before assigning access windows and probing the guest-facing Panthor driver.

```
[   29.053727]  arbiter.0: P0: Created gpu-subinstance with aw_mask=0x0
[   29.054166]  arbiter.0: P1: Created gpu-subinstance with aw_mask=0x0
[   29.054432]  arbiter.0: P2: Created gpu-subinstance with aw_mask=0x0
[   29.054632]  arbiter.0: P3: Created gpu-subinstance with aw_mask=0x0
```

When the `panthor_arbitration.ko` probes, partitions do not initially have assigned access windows.

Before probing `panthor.ko`, assign the `assigned_access_windows` mask.

The minimal bring-up example assigns all access windows to partition 0. Configure production systems to match the intended partitioning model and workload-isolation requirements.

panthor.ko

After you assign the slices and access windows, the guest-facing Panthor driver produces probe output.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Representative messages include:

```
[   61.676574] panthor 100000.gpu: [drm] Mali-G2-Pro id 0xf0805 major 0x0 minor 0x0
 status 0x0
[   61.890716] panthor 100000.gpu: [drm] CSF FW using interface v2.1.0.0, Features
 0x10 Instrumentation features 0x71
[   61.901255] [drm] Initialized panthor 1.7.0 for 100000.gpu on minor 2
```

These messages indicate that:

- Partition resources were assigned correctly.
- Firmware boot completed successfully.
- Driver initialization completed successfully.

A.8.4 Multi-partition GPU configuration

Multi-partition GPUs require additional configuration because slices, partitions, access windows, and you can independently assign resource groups.

panthor-system properties

Use the following `panthor-system` device-tree properties to configure resource-group assignments for AXI buses, partitions, slices, and access windows.

`am-resource-group-bus`
: Bitmask describing resource-group assignment to AXI buses.

`am-partition-resource-group`
: Bitmask describing partition assignment to resource groups.

`am-slice-resource-group`
: Bitmask describing slice assignment to resource groups.

`am-aw-resource-group`
: Bitmask describing access-window assignment to resource groups.

Arbitration properties

Use the following `panthor-arbitration` device-tree properties to configure default assigned access windows and slices for partitions.

`arm,default-slice-masks`
: Array of bitmasks that define the slices assigned to each partition. These masks must not overlap.

`arm,default-access-window-masks`
: Array of bitmasks that define the access windows assigned to each partition. These masks must not overlap.

Arbitration instances

The system creates 1 arbiter instance for each resource group that contains partitions or access windows.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Depending on the system configuration, arbiter instances can appear as:

- `arbiter.0`
- `arbiter.1`
- `arbiter.2`
- `arbiter.3`

The partitions visible through the `partitions/` SysFS hierarchy depend on the resource-group assignment specified by `am-partition-resource-group`.

SysFS entries

The `panthor_arbitration` driver exposes the following SysFS entries for viewing and configuring partition resources. Use these entries to assign slices and access windows to partitions and to verify the current partition configuration.

`/sys/bus/platform/devices/*.panthor-arbitration/arbiter.[0-3]/partitions/partition.[0-3]/assigned_slices`
: Bitmask of slices assigned to the partition.

Assign a slice to the parent resource group before assigning it to a partition within that resource group.

`/sys/bus/platform/devices/*.panthor-arbitration/arbiter.[0-3]/partitions/partition.[0-3]/assigned_access_windows`
: Bitmask of access windows assigned to the partition.

The default value for multi-partition systems is `0x0`.

Assign each access window to only one partition.

You must select the access-window masks to match the intended partitioning model, workload-isolation requirements, and guest layout.

The Panthor driver for an access window that is not assigned can fail to probe with `-ETIMEDOUT`.

Module parameters

`request_timeout`
: Time, in milliseconds, that the GPU remains assigned to an access window.

`yield_timeout`
: Time, in milliseconds, before the access window must yield the GPU. After this time, this window is forcibly closed.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A.9 Validate access-window switching

Validate that multiple guest-facing driver instances can request GPU ownership and use a shared GPU subinstance in turn.

Before you begin

Confirm that at least one access window can:

- Probe successfully
- Boot firmware
- Submit work successfully

Configure a second access window.

Procedure

1. Start workloads on both access windows.
   Enter the following code into the kernel:

   ```
   MALI_INSTANCE=0 <test_binary> &
   MALI_INSTANCE=1 <test_binary> &
   ```

2. Enable the access-window state-transition trace event:

   ```
   echo 1 > /sys/kernel/tracing/events/panthor_unstable/panthor_aw_set_state/enable
   ```

3. Display the trace log:

   ```
   cat /sys/kernel/tracing/trace
   ```

4. Verify that both access windows transition through ownership-request and ownership-grant states.

Results

The trace contains alternating ownership transitions for the participating access windows.

Example A-1: Example trace output

```
aw0-workload          [002] .....    78.870416: panthor_aw_set_state:
 dev=100000.gpu, old=READY, new=GPU_REQUEST
aw1-workload          [000] .....    78.870441: panthor_aw_set_state:
 dev=580000.gpu, old=READY, new=GPU_REQUEST
aw0-workload          [000] d.h..    79.034169: panthor_aw_set_state:
 dev=100000.gpu, old=GPU_REQUEST, new=GPU_GRANTED
aw0-workload          [000] d.h..    80.305833: panthor_aw_set_state:
 dev=100000.gpu, old=GPU_GRANTED, new=READY
aw0-workload          [005] .....    80.325374: panthor_aw_set_state:
 dev=100000.gpu, old=READY, new=GPU_REQUEST
aw1-workload          [000] d.h1.    80.625479: panthor_aw_set_state:
 dev=580000.gpu, old=GPU_REQUEST, new=GPU_GRANTED
aw1-workload          [000] d.h1.    81.885326: panthor_aw_set_state:
 dev=580000.gpu, old=GPU_GRANTED, new=READY
aw1-workload          [001] .....    81.901506: panthor_aw_set_state:
 dev=580000.gpu, old=READY, new=GPU_REQUEST
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A.9.1 Access-window switching trace states

The `panthor_aw_set_state` trace event records guest-side access-window state transitions during ownership transfer.

State meanings

`READY`
: The access window is initialized and ready to request GPU ownership.

`GPU_REQUEST`
: The access window has pending work and has requested GPU ownership from the arbiter.

`GPU_GRANTED`
: The arbiter has granted ownership of the GPU subinstance to the access window.

Interpreting trace output

During normal switching behavior:

1. The access window transitions from `READY` to `GPU_REQUEST` when work becomes available.
2. The arbiter grants GPU ownership, and the access window transitions from `GPU_REQUEST` to `GPU_GRANTED`.
3. The arbiter releases GPU ownership, and the access window transitions from `GPU_GRANTED` to `READY`.

Alternating `GPU_GRANTED` transitions between multiple access windows indicate that ownership transfer and arbitration are operating correctly.

For example:

- `aw0` transitions to `GPU_GRANTED`
- `aw0` releases ownership and returns to `READY`
- `aw1` transitions to `GPU_GRANTED`
- `aw1` releases ownership and returns to `READY`

This pattern demonstrates successful access-window switching on a shared GPU subinstance.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Appendix B pKVM worked example

This appendix provides a worked example of integration for a virtualized GPU system based on protected Kernel-based Virtual Machine (pKVM).

The example shows how to assign one GPU access window to a protected Virtual Machine (pVM), while privileged GPU control remains outside the guest. The deployment requires:

- The host owns privileged GPU control blocks.
- The guest owns only the assigned access-window register range.
- The hypervisor owns stage-2 mappings and device assignment.
- The system MMU applies the GPU-side memory-isolation policy that matches the guest assignment.

This appendix does not redefine the core virtualization architecture. For the architectural responsibilities that apply to all hypervisor-managed systems, see Hypervisor integration.

B.1 Scope of the pKVM example

This appendix describes a deployment in which:

- Privileged GPU control remains outside the guest.
- One access window is assigned to a protected Virtual Machine (pVM).
- The guest can access only its assigned GPU-facing interface.
- The protected Kernel-based Virtual Machine (pKVM) and the surrounding platform software manage system isolation, protected-memory assignment, and Virtual Machine (VM) level device assignment.

B.2 Why pKVM needs its own worked example

In any hypervisor-managed system, the architectural responsibilities are the same:

- Assign the correct access window to the guest.
- Retain ownership of privileged GPU pages outside the guest.
- Provide the required stage-2 isolation model for CPU and GPU access.
- Route interrupts to the correct owner.
- Explicitly expose protected memory where the guest must use it.

This appendix shows how the guidance in Hypervisor integration applies to a pKVM deployment.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

B.3 pKVM system deployment

This worked example uses a deployment in which:

- The privileged GPU control remains outside the guest.
- A protected Virtual Machine (pVM) is assigned one access window.
- The hypervisor manages device assignment and stage-2 isolation.
- The system MMU applies the GPU memory isolation policy for the assigned access window.

The deployment uses the following platform configurations and device assignment mechanisms:

Host configuration

The host system defines access windows in the device tree and configures system MMU support for these access windows. The host uses the Arm SMMU v3 driver named `arm_smmu_v3_pkvm`.

Host device-tree changes

The host device tree describes the access windows that can be used by the host or assigned to a pVM. Each access window has a distinct interrupt source in the device tree. For more information about the device tree, see Shared-partition GPU device-tree example.

For protected Kernel-based Virtual Machine (pKVM) integration, access windows use identity-mapped IOMMU handling. The following example shows two access windows that use separate StreamIDs:

```
&aw0 {
        iommu-idmapped;
        iommus = <&smmu_600 0>, <&smmu_600 1>;
};

&aw1 {
        iommu-idmapped;
        iommus = <&smmu_600 2>, <&smmu_600 3>;
};
```

Each access window uses StreamIDs that are separate from the StreamIDs used by other access windows.

pKVM device assignment node

A pKVM device assignment node identifies the access-window device that can be assigned to a pVM. The following example shows a device assignment node that references the access-window device aw1:

```
pkvm_device_assignment: pkvm-device-assignment {
        compatible = "pkvm,device-assignment";
        devices = <&aw1 0>;
};
```

The device assignment node lists the access-window device that is used for assignment.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Binding an access window to VFIO platform

Before the access window is assigned to a pVM, the host unbinds the access-window device from the host driver. Then, the host binds the device to the Virtual Function I/O (VFIO) framework using the vfio-platform driver.

To bind the access-window device to the vfio-platform driver, use the following sequence:

```
echo 400580000.gpu > /sys/bus/platform/drivers/panthor/unbind
echo vfio-platform > /sys/bus/platform/devices/400580000.gpu/driver_override
echo 400580000.gpu > /sys/bus/platform/drivers/vfio-platform/bind
```

Guest IOMMU behavior

The guest-visible IOMMU configuration uses passthrough behavior.

The guest driver uses the normal guest-visible GPU memory-management model. The hypervisor and the host system apply isolation through stage-2 mappings and system MMU configuration.

Android assignable device configuration

Android Virtualization Framework (AVF) uses an assignable device configuration file to list the host devices that can be assigned to a pVM.

The file is at:

```
/vendor/etc/avf/assignable_devices.xml
```

The following example shows one access-window device. In this example, the `sysfs_path` identifies the access-window device. The `dtbo_label` matches the Device Tree Blob Overlay (DTBO) used by the guest.

```
<devices>
        <device>
                <kind>gpu</kind>
                <dtbo_label>aw1</dtbo_label>
                <sysfs_path>/sys/bus/platform/devices/400580000.gpu</sysfs_path>
        </device>
</devices>
```

Guest VM DTBO overlay

The VM DTBO describes the GPU access window. The DTBO defines the access-window device that is visible to the pVM. The `dtbo_label` in the assignable device configuration file matches the DTBO used by the guest.

This example shows how the DTBO overlay enables the access-window node:

```
/dts-v1/;
/plugin/;

/ {
        fragment@0 {
                target-path = "/";
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
                __overlay__ {
                        aw1: aw1 {
                                compatible = "arm,mali-gen5-am";
                                status = "okay";
                        };
                };
        };
};
```

The following integration example shows a guest-visible access-window node with protected-memory assignment:

```
/dts-v1/;
/plugin/;

/ {
        fragment@0 {
                target-path = "/";

                __overlay__ {
                        gpu_fixed_clk: gpu-fixed-clock {
                                compatible = "fixed-clock";
                                #clock-cells = <0>;
                                clock-frequency = <100000000>;
                        };

                        reserved-memory {
                                #address-cells = <2>;
                                #size-cells = <2>;
                                ranges;

                                gpu_protected: gpu-protected@f5000000 {
                                        compatible = "arm,dma-buf-reserved";
                                        reg = <0x0 0xf5000000 0x0 0x02000000>;
                                };
                        };

                        dma_buf: dma-buf {
                                compatible = "simple-bus";

                                protected_heap: protected {
                                        compatible = "dma-buf,carveout";
                                        memory-region = <&gpu_protected>;
                                };
                        };

                        aw1: aw1 {
                                compatible = "arm,mali-gen5-am";
                                clocks = <&gpu_fixed_clk>;
                                clock-names = "gpuclk";
                                dma-buf-heap = <&protected_heap>;
                                interrupt-names = "IRQAW";
                                status = "okay";
                        };
                };
        };
};
```

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

B.4 Bring up and validate a pKVM deployment

Bring up and validate a protected Kernel-based Virtual Machine (pKVM)-based GPU deployment.

Before you begin

Before guest assignment starts, confirm that privileged services initialize correctly and can manage access-window control.

Procedure

1. Validate the privileged GPU stack outside the guest.
2. Assign 1 access window to a single protected Virtual Machine (pVM) and confirm that the guest can probe the guest-facing driver correctly.
3. Confirm that the protected-memory region required for firmware boot is exposed correctly.
4. Validate the access-window handshake, GPU request behavior, and ownership-transfer flow.
5. Validate teardown of the guest assignment and the privileged access-window reset operation.
6. Reassign the access window to another pVM after validation of the initial assignment lifecycle.

Results

This procedure validates a complete assignment lifecycle, including protected-memory configuration, ownership transfer, teardown, and reassignment.

B.5 Access-window assignment and reassignment

Some protected Kernel-based Virtual Machine (pKVM) deployments require access windows to be reassigned between protected Virtual Machines (pVMs) over time. Reassigning an access window is different from transferring ownership of a GPU subinstance between guests. The reassignment principles described in Access-window reassignment apply in pKVM deployments.

Before you assign an access window to a different guest, the privileged software must perform an access-window reset. This reset clears the guest-specific state that is associated with that window and preserves the ownership and isolation model.

This example covers:

- Assignment of an access window to a pVM
- Teardown of that assignment
- Privileged reset of the access window
- Reassignment of the access window to a different pVM

B.6 Guest-visible device model in pKVM

From the guest perspective, the device model is simple:

- A single assigned access-window register block

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- A single guest-facing interrupt
- Any guest-visible protected-memory description required for firmware boot or protected-mode operation

The platform-specific mechanics that protected Kernel-based Virtual Machine (pKVM) uses to deliver that assignment can vary. The key architectural requirement is consistency. The guest-visible model must remain consistent with the ownership model that the main hypervisor-integration chapter describes.

B.7 Protected-memory handling in pKVM

Protected-memory handling is a key part of protected Kernel-based Virtual Machine (pKVM) deployments. The protected-memory assignment principles described in Protected-memory mapping apply in pKVM deployments.

Firmware boot and protected-mode operation can depend on explicit assignment of reserved memory to a guest.

A typical pKVM deployment can include:

- A protected-memory region required for CSF firmware boot
- Additional protected-memory regions assigned to protected-mode applications or validation workloads
- Platform-specific mechanisms used to assign those regions to a protected Virtual Machine (pVM)
- A guest-visible description of the assigned memory regions

In a typical flow, privileged software reserves the regions, applies pKVM assignment controls, and then exposes the assigned ranges to the guest through the expected guest-visible interface. The guest can then initialize firmware and protected-mode workloads by using only the regions that were assigned.

If a guest is expected to use protected memory, the platform must assign that memory explicitly.

Assigning an access window to a guest does not automatically provide access to protected memory.

For integration and validation, confirm these points:

- The guest cannot access protected-memory regions that were not assigned to it.
- Firmware boot succeeds only when required protected-memory regions are present.
- Access-window assignment and protected-memory assignment are tracked as separate controls.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Appendix C Porting Guide

This appendix is intended for the following use cases:

- Port the current Linux reference stack to another operating system
- Replace part of the stack with a custom implementation
- Write a new virtualization stack that preserves the same architectural model

It is intentionally implementation-oriented, and does not restate the whole design. Instead, it identifies the software-visible contracts that a replacement or ported stack must preserve and the areas where you are free to diverge.

C.1 Scope of porting effort

Porting efforts can vary significantly in scope and complexity. Before planning a port, determine which parts of the software stack you must replace or adapt. The scope of the port affects which interfaces, responsibilities, and architectural contracts you must preserve.

Common porting scenarios include:

- Porting only the guest-facing access-window driver behavior to another operating system
- Retaining the guest-side model while replacing the host-side virtualization modules
- Retaining the host-side model while replacing the guest-side operating system
- Replacing the full virtualization stack while preserving the hardware architecture

Defining the scope of the porting effort early helps identify the relevant interfaces and implementation requirements. Some interfaces apply only to the guest-facing path. Other interfaces apply only to privileged host-side ownership and arbitration.

For example, if you keep the guest-side contract stable, focus on preserving register behavior, interrupt expectations, and access-window timing visible to guest software. If you replace host-side modules, preserve arbitration ownership boundaries, reassignment behavior, and isolation guarantees expected by the hypervisor and firmware.

Understanding the scope of the work helps determine:

- Which software-visible contracts must remain compatible
- Where implementation-specific changes are acceptable

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

C.2 Two porting strategies

Most porting efforts follow 1 of 2 strategies:

Compatible replacement

Preserve the external behavior of the current Linux reference stack. This approach enables existing guest-side and host-side components to interoperate with the new implementation.

Platform-native redesign

Preserve the hardware ownership model and architectural contracts. Replace Linux-specific driver split, control surfaces, and OS service abstractions with platform-native mechanisms.

Use compatible replacement when you want to retain interoperability with the existing reference stack or preserve debug comparability. Use platform-native redesign when the target OS uses a very different driver model, for example an RTOS or a microkernel-based system.

C.3 Architectural contracts

Any custom implementation, whether Linux-based or not, must preserve the following architectural contracts:

- One guest-visible driver instance is associated with one access window.
- The guest-visible path can request GPU ownership and receive stop requests.
- The privileged side controls assignment, handover, and timeout policy.
- The hypervisor, or equivalent privileged platform software, remains respondible for isolation.
- Runtime suspend and resume operations still follow the same ownership graph.
- Protected memory, where used, is assigned explicitly rather than discovered implicitly.

These contracts define the required behavior. The source-file structure, kernel object model, and operating system service model can vary between implementations.

C.4 Porting interfaces

The current source tree exposes several concrete interface points that are useful when planning a port or replacement.

When you port the virtualization stack, focus on preserving the architectural contracts rather than reproducing the Linux implementation. The source code structure, driver model, and operating system services can change. However, the ownership, communication, scheduling, and isolation behavior must remain unchanged.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

C.4.1 Arbiter-VM protocol

The Arbiter-VM protocol defines the control messages exchanged between the guest-side driver and the privileged arbitration side.

Protocol definition

The protocol definition is located in:

`drivers/gpu/drm/panthor/include/am/panthor_arb_vm_protocol.h`

Required messages

A compatible implementation must support the following messages:

- `ARB_VM_INIT`
- `VM_ARB_INIT`
- `VM_ARB_GPU_REQUEST`
- `ARB_VM_GPU_STOP`
- `VM_ARB_GPU_STOPPED`

Compatability requirements

These messages form part of the control channel between the guest-side driver and the privileged arbitration side. They do not describe the whole virtualization interface by themselves, but they are a required part of it.

A replacement implementation does not need to preserve the Linux source layout or internal module boundaries. It must preserve the protocol semantics, ordering expectations, and state transitions represented by these messages.

Failure to preserve these semantics can cause incorrect initialization, GPU ownership handover, or stop-processing behavior.

C.4.2 Core contracts that must be preserved

Any custom implementation, whether Linux-based or not, must preserve the following architectural contracts.

One guest-visible driver instance per access window
: Each guest-visible driver instance must be associated with a single access window.

GPU ownership request and stop handling
: The guest-visible path must be able to request GPU ownership and respond to stop requests.

Privileged ownership of arbitration policy
: The privileged side must remain responsible for assignment, ownership handover, and timeout policy.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Ownership of isolation

The hypervisor, or equivalent privileged platform software, must remain responsible for isolation.

Suspend and resume ordering

Runtime suspend and resume operations must follow the established ownership hierarchy.

Protected-memory assignment

Protected memory, where used, must be assigned explicitly rather than discovered implicitly.

Implementation flexibility

These architectural contracts must be preserved regardless of implementation details.

The following implementation choices can vary:

- Source-file organization
- Kernel object model
- Operating-system service abstractions

C.4.3 What the guest side observes

In the Linux reference implementation, the guest-side access-window driver relies on several sources of information to determine ownership and state transitions.

The guest-side interface includes:

- Incoming and outgoing message registers for the software protocol
- Access-window interrupts:
  - Message arrival
  - Window opened
  - Window closed
  - Invalid access
- Access-window status information, including whether the window is currently open

The access-window interface also provides the transitional `WINDOW_OPENING` event. Not all implementations need to model this event as a separate high-level state.

The guest does not determine GPU ownership from protocol messages alone. Ownership state is established through a combination of protocol messages and access-window events.

For example:

- The guest sends `VM_ARB_GPU_REQUEST` to request ownership.
- The guest later observes `WINDOW_OPENED` to confirm that the GPU is connected to the access window.
- The guest receives `ARB_VM_GPU_STOP` when ownership must be released.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- The guest later observes `WINDOW_CLOSED` to confirm that ownership has been removed.

In the preferred yield path, ownership transfer follows this sequence:

1. The arbiter issues `YIELD_NOW` or `YIELD_IDLE` through partition control.
2. The GPU and MCU receive the corresponding yield interrupt.
3. Firmware quiesces activity and issues `YIELD_DONE`.
4. The Access Manager closes the access window and raises `WINDOW_CLOSED`.
5. The partition completes reset and the privileged side observes `PARTITION_RESET_DONE`.

Implementations must consider both protocol messages and access-window events when tracking ownership state. Treating protocol messages as the sole source of truth can result in incorrect ownership-transfer handling.

C.4.4 What the privileged side observes

In the Linux reference implementation, the privileged arbitration stack relies on 2 hardware-facing paths:

- Resource-group communication for per-access-window messages
- Partition-control commands, status, and reset notifications for actual ownership transfer

The resource-group side receives and sends access-window messages. The partition-control side opens and closes windows, issues `YIELD_NOW` and `YIELD_IDLE` requests, and detects completion of reset and ownership transitions.

These interfaces have different roles. The message path communicates guest requests and status information. The partition-control path performs the hardware operations that transfer ownership.

The hardware interfaces provide the following capabilities:

- Message-pending status for each access-window message channel
- A partition state machine that reports states such as reset, window opening, window open, and window closed

A ported implementation must preserve both interfaces. It must preserve the message semantics and the status-driven ownership-transfer sequence. A message protocol alone is not sufficient because ownership transfer depends on commands, status information, and interrupts as well as messages.

C.4.5 Porting implication

When you port or replace the privileged stack, treat the arbiter-to-VM interface as a single signaling model.

This signaling model has 4 parts:

- A bidirectional message protocol

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- A privileged command path for ownership changes
- A status path that reports ownership state and transfer progress
- An interrupt path that notifies each side about relevant events

These parts work together to transfer ownership correctly.

A replacement implementation does not need to match the Linux software architecture or driver structure. It does need to preserve the signaling behavior defined by the model.

Ownership transfer depends on coordinated interaction between messages, commands, status information, and interrupts. Preserve these semantics even when you use different operating-system services or software components.

C.4.6 Arbitration-side public interfaces

The current arbitration-side public interface definitions are located in:

```
drivers/gpu/drm/panthor/arbitration/panthor_arbiter.h
```

These interfaces expose the software contracts between the guest-side driver, the arbitration layer, and the privileged ownership model.

Interface responsibilities

Use the following responsibilities to identify the interfaces that the replacement implementation must provide.

VM registration
: Registering and tracking guest instances that participate in arbitration.

GPU request notification
: Receiving requests for GPU ownership from guest-side drivers.

GPU stopped notification
: Receiving notification that a guest-side stop or yield sequence has completed.

VM assignment callbacks
: Assigning and unassigning access windows, GPU subinstances, or partitions.

Yield callbacks
: Initiating ownership-transfer operations such as immediate or idle-time yield.

Repartition callbacks
: Updating partition-resource assignments on partition-capable systems.

Resource-group queries
: Obtaining information about resource groups and their current configuration.

Porting considerations

These interfaces are useful because they make the Linux reference implementation's software contracts visible.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A replacement implementation does not need to preserve the same C interfaces. However, it should preserve the underlying responsibilities that these interfaces represent.

In particular, a replacement implementation should provide mechanisms for:

- Guest registration and lifecycle management
- GPU ownership requests and grants
- Stop, yield, timeout, and recovery handling
- Access-window, partition, or GPU-subinstance assignment
- Resource-group management where supported

The implementation can expose these functions through different operating-system services or software abstractions. However, it should preserve the ownership, scheduling, and communication semantics that the virtualization architecture defines.

C.5 Porting the guest-side driver model

If you port the guest-facing access-window path to another operating system, the new driver must preserve the functional behavior of the Linux reference implementation.

The guest-side implementation is responsible for the following functions:

- Ownership requests
- Stop handling
- Access-window events
- State management
- Integration with the ownership-transfer model

It must also preserve any protected-memory requirements used by the platform.

A non-Linux port does not need to preserve the Linux source layout or operating-system abstractions. It does need to preserve the guest-side behavior, ownership-transfer semantics, and software-visible contracts that other components depend on.

The following topics describe the guest-side responsibilities, state handling, interrupt handling, and protected-memory requirements in more detail.

C.5.1 Porting the guest-side driver model

If you port the guest-facing access-window path to another operating system, the new driver must preserve the functional behavior of the Linux reference implementation.

The guest-side implementation is responsible for ownership requests, stop handling, access-window events, state management, and integration with the ownership-transfer model. It must also preserve any protected-memory requirements used by the platform.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A non-Linux port does not need to preserve the Linux source layout or operating-system abstractions. It does need to preserve the guest-side behavior, ownership-transfer semantics, and software-visible contracts that other components depend on.

The following topics describe the guest-side responsibilities, state handling, interrupt handling, and protected-memory requirements in more detail.

C.5.2 Guest-side state handling

The guest-side implementation requires an internal state model to track ownership requests, ownership grants, and ownership-transfer operations.

At a minimum, the guest-side implementation should distinguish between:

- Handshake not complete
- Ready to request ownership
- GPU ownership requested
- GPU ownership granted
- Stopping
- Stopped or unassigned

These states represent the major phases of the guest-side ownership lifecycle. The exact implementation can vary, but the underlying state transitions should remain clear and well defined.

A guest-side implementation that does not maintain a clear state model can experience lost requests, incorrect ownership tracking, duplicate grant handling, or incorrect recovery following a `WINDOW_CLOSED` event.

C.5.3 Guest-side interrupt handling

The guest-side implementation relies on access-window interrupts to track ownership changes and protocol activity.

A guest-side port must preserve the meaning of the key access-window interrupts:

- Message interrupt
- Window opened
- Window closed
- Invalid access

These interrupts provide information that is not available through protocol messages alone. The guest-side implementation uses them to detect message arrival, confirm ownership changes, and identify invalid access attempts.

If the target platform also supports the asynchronous yield mechanism, the guest-side design must also handle the relationship between protocol messages and `YIELD_NOW`, `YIELD_IDLE`, and `YIELD_DONE`.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

It is important to preserve the distinction between:

- A guest requesting ownership
- The hardware granting ownership by opening the window
- A guest receiving a stop request
- The hardware removing ownership by closing the window

These are separate events in the Panthor ownership-transfer model and should remain separate in a compatible implementation.

C.5.4 Guest-side protected memory

Some platforms use protected memory for CSF firmware boot or protected workloads. In those cases, the guest-side implementation must provide access to the protected-memory resources assigned to it.

The Linux reference implementation uses a protected DMA heap identified through `protm_heap_name`. A non-Linux port does not need to preserve the DMA heap API or allocation mechanism.

A non-Linux port must provide equivalent protected-memory functionality so that:

- The guest can discover its assigned protected-memory region.
- The guest can allocate memory from that region when required.
- The system continues to enforce the associated security policy.

C.6 Porting or replacing the privileged virtualization stack

When you port or replace the privileged virtualization stack, you can reorganize the software architecture. However, you must preserve the core responsibilities of the reference architecture.

The privileged implementation must continue to provide:

- System-control services for Access Manager configuration, privileged interrupt handling, and state management
- Arbitration services for ownership control, scheduling, yield handling, timeout handling, recovery, and access-window management
- Governor services for DVFS and dynamic core-scaling policy
- Assignment services for assigning GPU subinstances or partitions to VMs and, where supported, repartitioning GPU resources

You can replace the reference implementations and scheduling policies with platform-specific alternatives. However, the ownership, assignment, power-management, and virtualization contracts defined by the architecture must remain unchanged.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

C.6.1 System-control role

`panthor_system.ko` must be the first driver to probe during the boot sequence. It must initialize the clocks and regulators first to enable valid hardware access.

A replacement for `panthor_system.ko` must provide these capabilities. For more information, see Probe and module load order.

Privileged AM system register handling
: Access to and management of privileged Access Manager system registers.

Privileged interrupt handling
: Handling of privileged interrupts generated by the system block.

Suspend and resume support
: Saving and restoring privileged system configuration across suspend and resume operations.

Assignment-block support
: Any required assignment-block functionality for the target product line.

C.6.2 Arbiter role

A replacement for `panthor_arbitration.ko` must provide the following capabilities:

VM model
: A virtual-machine model associated with access-window identity.

Arbitration policy
: Arbitration policy to select and manage GPU ownership between competing access windows.

Stop and yield handling
: Stop-request and ownership-yield handling.

Timeout handling
: Ownership-transfer timeout detection and handling.

Recovery
: Recovery when orderly yield completion does not occur.

Assignment control
: Control of access-window assignment to GPU subinstances or partitions.

Resource-group messaging
: Functionality equivalent to resource-group message handling.

Partition-control handling
: Functionality equivalent to partition-control commands, status monitoring, and interrupt handling.

Scheduling policy

The round-robin scheduler in the Linux reference implementation is only an example policy.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

You can replace the scheduling algorithm, provided that the implementation preserves the external behavioral contract for guest requests, ownership grants, stop handling, and timeout processing.

C.6.3 Governor role

The governor role is responsible for GPU performance and power-management policy. This includes utilization monitoring, Dynamic Voltage and Frequency Scaling (DVFS) control, and optional dynamic core scaling. Any replacement implementation must preserve the platform's power-management model and software-stack integration requirements.

Governor functions

Use the following functions to identify the capabilities that the replacement governor must provide.

Utilization measurement
: The governor can use the `AM_GOV_IPA_CONTROL` register page to sample GPU utilization.

DVFS control
: DVFS management.

Dynamic core scaling
: Optional support for dynamic GPU core scaling.

Stack integration
: Coordination with the surrounding software stack and the power-management ownership model.

Porting considerations

You can replace the governor more freely than the guest-arbiter messaging contract.

However, any replacement must remain compatible with the platform power-management ownership model and the ordering requirements of the surrounding software stack.

C.6.4 VM assignment and repartition roles

The arbitration interfaces distinguish between:

- Assignment of a GPU subinstance or partition to a VM
- Repartitioning of slices between partitions

In the Linux headers:

- `vm_assign_ops` defines assignment-side callbacks such as `assign_vm`, `unassign_vm`, `get_assigned_vm`, and optional `yield_now` or `yield_idle`
- `repartition_ops` defines `assign_slices`

If the target system supports only time-sliced operation and not partitioning, the repartition side can be omitted. If the target system supports partition-capable hardware, that capability must be preserved in the privileged implementation.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

C.6.5 VM assignment and repartition interfaces

The virtualization model distinguishes between assignment and repartitioning.

Assignment controls which GPU subinstance or partition is associated with a virtual machine (VM). Repartitioning controls how slices are distributed between partitions.

These functions are independent. A platform can support assignment without repartitioning.

Systems that support only time-sliced execution can omit repartitioning functionality. Systems that support partition-capable hardware must preserve an equivalent repartitioning capability in the privileged implementation.

C.7 Linux service equivalents for other operating systems

A non-Linux port must provide functional equivalents for the Linux facilities on which the reference implementation relies.

Interrupt handling
: Interrupt registration and deferred interrupt handling.

Timeout management
: Timers used for request and yield timeouts.

Deferred execution
: Workqueues or another mechanism for deferred processing.

Blocking waits
: Wait mechanisms for handshake completion and stop completion.

Power management
: Runtime power-management coordination.

Dependency management
: An equivalent mechanism for device dependency ordering.

Hardware description
: A hardware-description mechanism, such as device tree or a platform-specific equivalent.

Protected-memory allocation
: Protected-memory allocation support, where required.

Memory management
: System MMU or IOMMU integration.

Clock and regulator control
: Management of clocks and regulators when owned by software.

Tracing and logging
: Logging or tracing sufficient to diagnose ownership-transfer and handover failures.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Porting considerations

A port does not need a one-to-one API match with Linux.

However, it does need a one-to-one functional match for the behaviors that these facilities provide.

C.7.1 Identify operating-system service equivalents

Before you begin a port, identify platform-native equivalents for the Linux services that the reference implementation uses.

Linux service mappings

Use the following mappings to identify equivalent services in the target operating system.

Linux IRQ top half and threaded or deferred handling
: Equivalent interrupt-handling and deferred-work model in the target operating system

Linux workqueues and timers
: Equivalent mechanism for scheduler ticks, timeout handling, and cleanup outside interrupt context

Linux runtime PM and device links
: Equivalent mechanism for dependency-aware suspend and resume ordering

Linux device tree and driver probe
: Equivalent source of platform description and dependency discovery

Linux DMA heaps or protected-memory allocators
: Equivalent protected-memory allocation and ownership model

Linux tracepoints, debugfs, sysfs, and log output
: Equivalent observability model for handshake failures, stop latency, timeout handling, and recovery

Porting considerations

The target operating system does not need to provide the same APIs as Linux. However, it must provide equivalent functionality for the behaviors that the virtualization stack requires.

If the target operating system cannot provide a suitable equivalent for one of these areas, the port typically requires an architectural adaptation layer rather than direct code translation.

C.8 Device tree, platform description, and configuration

The Linux reference implementation uses device tree to describe platform configuration and hardware dependencies.

Device-tree information

The device tree provides:

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Register ranges
Hardware register address ranges.

Interrupt descriptions
Interrupt definitions and routing information.

Clock and regulator wiring
Clock and regulator dependencies.

OPP and IPA-model data
Operating performance point and power-model information.

Dependency graphs
Device dependencies through `deps` supplier links.

Assignment data
Assignment configuration where the assignment block is present.

Porting requirements

A port to another operating system does not need to preserve the device-tree syntax. It does need to preserve the information represented by the device tree.

The target platform must provide a way to describe:

- Which access windows belong to which guest-visible driver instances
- Which privileged modules own specific register pages
- Probe and power-management ordering
- Governor hierarchy configuration
- Initial assignment or repartition configuration

C.9 RTOS and microkernel considerations

A Real-Time Operating System (RTOS) or microkernel port typically requires additional design work in three areas:

Concurrency model
The Linux reference stack assumes sleeping waits, deferred work, and background timeout handling. A smaller operating system might require a more explicit event-driven state machine.

Privilege split
The privileged modules might not map naturally to separate kernel drivers and could instead become secure-monitor services, a host daemon, or a combined privileged driver.

Configuration model
Device tree, sysfs, and module parameters are Linux-specific mechanisms. A non-Linux implementation still requires equivalent configuration and debug control points, whether these are implemented through static tables, shell commands, or OEM service interfaces. For more information, see Probe and module load order.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Make these design decisions early in the porting effort. An RTOS or microkernel port that attempts to preserve Linux internal structure too closely often becomes more difficult to validate and maintain.

C.10 What a partner can change freely

A partner implementation can modify these areas without affecting the architectural contracts of the virtualization stack.

Scheduler policy
: The arbitration and scheduling policy used to select GPU ownership.

Internal object model
: Internal software structures and object relationships.

Driver and library organization
: The split between drivers, services, and helper libraries.

Operating-system abstractions
: Platform-specific service interfaces and operating system integration.

Configuration interface
: User-facing configuration, control, and management mechanisms.

Porting considerations

These areas are implementation choices. A replacement implementation can change them, provided that it continues to satisfy the required architectural contracts and externally visible behavior.

C.11 What a partner must not break

A partner implementation must preserve the following architectural contracts.

Guest-visible ownership semantics
: Ownership requests, grants, and ownership-transfer behavior must remain consistent with the architectural model.

Interrupt semantics
: The meaning and handling of interrupts must remain unchanged.

Handshake and GPU request flow
: Initialization, handshake, and GPU ownership-request behavior must remain compatible.

Stop and timeout semantics
: Stop handling, timeout handling, and recovery behavior must remain consistent.

Ownership of isolation
: The hypervisor or equivalent privileged platform software must remain responsible for isolation.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Power-management ordering

Runtime suspend, resume, and ownership-transfer ordering must be preserved.

Protected-memory ownership and assignment

Protected-memory assignment and ownership rules must remain unchanged.

Porting considerations

These behaviors form the software-visible contract that other layers of the system depend on. A replacement implementation can change its internal architecture, but it must preserve these externally visible semantics.

C.12 Validate a port

Validate the ported or replacement virtualization stack.

Procedure

1. Probe and initialize all required modules in the intended dependency order.
2. Complete the arbiter-VM handshake successfully.
3. Request GPU ownership from the guest side and receive a grant.
4. Exercise orderly stop and yield behavior.
5. Exercise timeout and recovery behavior.
6. Validate runtime suspend and resume ordering.
7. Validate protected-memory boot requirements for CSF firmware, where used.
8. Validate access-window reassignment and reset behavior, where supported.
9. Validate partition reassignment and slice reassignment, where supported.
10. Verify that tracing or logging is sufficient to diagnose lost requests, failed stop handling, and `GPU_LOST` paths.

Related information

Probe and module load order on page 68

C.13 Scope of the porting guide

When you port or replace part of the virtualization stack, focus on:

- The guest-facing access-window driver model
- The privileged arbitration model
- The signaling model between the guest and the arbiter
- The ownership-transfer model, including ordering, timeout, and recovery behavior

Implementation-specific details are outside the scope of this guide.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Appendix D Arbiter-VM Protocol Appendix

This appendix summarizes the arbiter-to-guest protocol used on the access-window messaging path. It is intended primarily as a debugging and instrumentation reference. It does not describe a partner customization interface, and it is not normally expected to modify the protocol.

For debugging, read this protocol alongside the asynchronous yield path. The preferred ownership-transfer sequence is:

```
YIELD_NOW or YIELD_IDLE → YIELD_DONE → WINDOW_CLOSED
```

In this flow, firmware completes the yield operation and notifies the Access Manager asynchronously in hardware. This removes the direct dependency between yield completion and CPU scheduling latency.

The `ARB_VM_GPU_STOP` and `VM_ARB_GPU_STOPPED` messages belong to the legacy software-side path, which remains for compatibility and implementation reasons.

It does not reproduce the full behavior of the reference source code line by line. When using the supplied reference stack, treat the source code as the authoritative reference for implementation details.

This appendix focuses on:

- Message purpose and direction
- The relationship between protocol messages and access-window interrupts
- The difference between the preferred asynchronous yield path and the legacy software stop path
- The main guest-visible states used to interpret logs and traces

For systems that use a custom arbiter-side implementation or a non-Linux guest-side implementation, this appendix explains the protocol contract.

Use the implementation source code as the primary reference for detailed state-machine behavior, retry logic, and implementation-specific sequencing.

D.1 Purpose of the protocol

The protocol provides a lightweight control channel between the guest-side driver and the arbiter-side software. It complements the hardware-controlled ownership-transfer flow by allowing software to coordinate initialization, GPU access requests, and stop notifications.

Read this protocol together with the access-window interrupt model:

- Message interrupts indicate that a new protocol message has been received.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- Window-open and window-close interrupts indicate changes in hardware ownership.
- Invalid-access interrupts indicate that software attempted to use an access window while the corresponding GPU subinstance was inaccessible through that path

In practice, protocol messages and access-window interrupts are required to understand the full software flow.

Not all parts of the protocol have the same architectural importance. GPU request and initialization messages are part of the normal software model. By contrast, `GPU_STOP` and `GPU_STOPPED` belong to the legacy software-driven yield path rather than the preferred firmware-driven yield path.

D.2 Versioning

The reference software stack currently supports protocol version `3`. The minimum supported protocol version is also `3`.

In partition-capable systems, the initialization exchange can include additional partition-related information, for example the maximum shader core count visible through the access window.

D.3 Message directions

The protocol uses a fixed set of message identifiers.

Arbiter to VM

- `ARB_VM_GPU_STOP`
- `ARB_VM_INIT`

VM to Arbiter

- `VM_ARB_INIT`
- `VM_ARB_GPU_REQUEST`
- `VM_ARB_GPU_STOPPED`

D.4 Message meanings

Describes the purpose and behavior of each protocol message.

ARB_VM_INIT

Initiates or completes the handshake between the arbiter side and the guest side.

This message carries:

- The protocol version
- An acknowledge bit
- Optional partition capability information in partition-capable systems

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

At system level, this message establishes or re-establishing the messaging relationship between the arbiter side and the guest-side driver for a given access window.

VM_ARB_INIT

This message is the guest-side counterpart to `ARB_VM_INIT`. It is used to complete the handshake as part of that initialization flow.

At system level, this message indicates that the guest-side driver is ready to communicate using the agreed protocol version.

VM_ARB_GPU_REQUEST

Requests GPU access for a specific access window.

The message includes a small priority field. The platform-specific interpretation of that field depends on the arbiter-side implementation.

At system level, this message indicates that the guest has pending work and requests ownership of the GPU subinstance.

ARB_VM_GPU_STOP

Requests that the guest-side driver stop using the GPU.

In the current implementation, receipt of this message:

- Transitions the access-window state into a stopping state
- Queues suspend handling in the kernel driver

Architecturally, this message belongs to the legacy software-side stop path rather than the preferred asynchronous yield path.

The asynchronous yield path uses:

- `YIELD_NOW`
- `YIELD_IDLE`
- `YIELD_DONE`

`ARB_VM_GPU_STOP` is the guest-visible stop request used by the older software-driven path.

VM_ARB_GPU_STOPPED

Indicates that the guest-side driver has finished using the GPU.

At system level, this message informs the arbiter that the access window can be treated as stopped from the guest-driver perspective.

Architecturally, this message also belongs to the legacy software-side stop path. It acts as the guest-driver acknowledgment in flows that do not rely exclusively on firmware-driven `YIELD_DONE` completion.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

In the current implementation, this message remains active and is used during:

- Suspend paths
- Unplug paths
- Transitions into the `GPU_STOPPED` state

When debugging, do not treat `VM_ARB_GPU_STOPPED` as equivalent to `YIELD_DONE`.

- `YIELD_DONE` is the completion signal used by the asynchronous yield path
- `VM_ARB_GPU_STOPPED` is the guest-driver completion message used by the legacy software path

D.5 Typical protocol flow

This topic summarizes the typical ownership-transfer flow between the arbiter side and the guest side.

Preferred asynchronous-yield flow

1. The arbiter side and guest side complete the `INIT` handshake.
2. The guest side sends `VM_ARB_GPU_REQUEST` to request GPU ownership.
3. The access window opens and the guest becomes the active owner.
4. The arbiter requests yield through the asynchronous yield path, for example with `YIELD_NOW` or `YIELD_IDLE`.
5. Firmware completes the yield sequence and signals `YIELD_DONE`.
6. The Access Manager closes the access window and raises `WINDOW_CLOSED`.
7. The arbiter continues ownership transfer and can grant the GPU to the next access window.

Legacy software-driven flow

1. The arbiter sends `ARB_VM_GPU_STOP` to the guest-side driver.
2. The guest-side driver stops GPU use and sends `VM_ARB_GPU_STOPPED`.
3. The arbiter manually closes the access window. Closing the access window raises `WINDOW_CLOSED` and completes the guest-side stop flow.
4. The arbiter continues ownership transfer and can grant GPU ownership to the next access window.

Why this distinction matters

The protocol supports 2 stop mechanisms:

- The preferred asynchronous yield path (`YIELD_NOW`, `YIELD_IDLE`, `YIELD_DONE`)
- The legacy software-driven path (`ARB_VM_GPU_STOP`, `VM_ARB_GPU_STOPPED`)

When debugging ownership transfer, identify which path the system is using before you interpret message and interrupt timing.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

D.6 Guest-side access-window states

The guest-side access window can transition through the following states.

State meanings

INIT
: Initial state before the arbiter/VM handshake completes.

READY
: Handshake complete. The access window can request GPU ownership.

GPU_REQUEST
: Guest requested GPU access and is waiting for a grant.

GPU_GRANTED
: Access window currently owns the GPU subinstance.

GPU_STOP
: Guest received a stop request and is transitioning toward suspension.

GPU_STOPPED
: Guest-side driver stopped GPU use and notified the arbiter.

Debugging guidance

These states can help identify where ownership transfer is failing, such as during handshake, request, execution, stop handling, or completion of the stop sequence.

D.7 Relationship to access-window interrupts

Ownership transfer depends on access-window interrupts and asynchronous yield signals.

Access-window interrupts

MESSAGE
: New arbiter message available. `WINDOW_OPENED`
: Access window was granted access to the GPU subinstance. `WINDOW_CLOSED`
: Access window was closed. `INVALID_ACCESS`
: Software attempted to access the closed portion of the access window.

Correlating protocol and hardware events

When debugging, correlate protocol messages with access-window interrupts.

For example, the guest was suspended due to a `ARB_VM_GPU_STOP` message, while `WINDOW_CLOSED` confirms that hardware handover progressed.

It is also useful to correlate these events with the asynchronous yield sequence:

YIELD_NOW
: Request to stop as quickly as possible. `YIELD_IDLE`

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Request to yield after the current queue drains and the GPU becomes idle. `YIELD_DONE` Completion signal that yield finished and ownership transfer can continue.

Key distinction

The preferred ownership-transfer mechanism is the asynchronous yield path.

`GPU_STOP` and `GPU_STOPPED` belong to the legacy software-driven path.

For reliable debugging, analyze message, interrupt, and yield timelines together.

D.8 Debugging notes

Use this protocol to diagnose communication, ownership-transfer, and shutdown issues between the arbiter and guest-side driver.

Common debugging scenarios include:

- Handshake failures between the arbiter and guest driver
- Guests that request GPU ownership but never receive it
- Guests that receive a stop request but do not complete the firmware-driven yield path or the legacy software stop path
- Mismatches between software-visible stop handling and hardware-visible window-close events
- Timeout and device-lost paths

When instrumentation is available, use a combined timeline to get the clearest view of system behavior.

Correlate these events:

- Arbiter-to-VM and VM-to-arbiter protocol messages
- Access-window interrupts
- Firmware yield activity
- Guest-driver suspend and resume events

This correlation helps you identify whether a failure starts in arbitration policy, message handling, firmware yield processing, or later reset and ownership-transfer stages.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported,

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at https://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in Arm documents.

Product status

All products and services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is Final, that is for a developed product.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0100-01	30 September 2026	Non-Confidential	First release for version 1.0

Change history

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in Document release information on page 117.

Table 2: Issue 0100-01

Change	Location
First release of version 1.0.	-

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
italic	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
`monospace`	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
`monospace underline`	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
`<and>`	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: `MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>`
SMALL CAPITALS	Terms that have specific technical meanings as defined in the *Arm® Glossary*. For example, **IMPLEMENTATION DEFINED**, **IMPLEMENTATION SPECIFIC**, **UNKNOWN**, and **UNPREDICTABLE**.



We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential



You are at risk of causing permanent damage to your system or your equipment, or harming yourself.



This information is important and needs your attention.



A useful tip that might make it easier, better or faster to perform a task.



A reminder of something important that relates to the information you are reading.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Arm® Mali™ GPU Valhall and Earlier Virtualization Guide	102104	Non-Confidential
System Control and Management Interface	–	Non-Confidential

Non-Arm resources	Document ID	Organization
ARM System Control and Management Interface (SCMI)	–	Xen
Android Virtualization Framework (AVF) overview	–	Android
LLVM project	–	https://www.llvm.org
OpenGL ES 2.0 Programming Guide (August 2008), Addison-Wesley Professional	ISBN 0-321-50279-5	-
OpenGL ES 2.0 Specification	–	https://www.khronos.org
OpenGL ES 3.0 Programming Guide (March 2014), Addison-Wesley Professional	ISBN 0-321-93388-5	-
OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 2 (5th Edition, 2005), Addison-Wesley Professional	ISBN 0-321-33573-2	-
OpenGL Shading Language (2nd Edition, 2006), Addison-Wesley Professional	ISBN 0-321-33489-2	-
Protected KVM (pKVM)	–	-
The C standard: ISO/IEC 9899:1990 GPU OpenGL ES Information technology - Programming Language C (1990). ISO/IEC.	–	-

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential